

Chương 10. Giới thiệu về Mạng nơ-ron nhân tạo với Keras

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 10 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Chim chóc đã truyền cảm hứng cho chúng ta bay lượn, cây ngưu bàng đã truyền cảm hứng cho Velcro, và thiên nhiên đã truyền cảm hứng cho vô số phát minh khác. Vì vậy, việc tìm kiếm nguồn cảm hứng từ cấu trúc não bộ để xây dựng một cỗ máy thông minh dường như là điều hoàn toàn hợp lý. Đây chính là logic đã tạo nên mạng lưới thần kinh nhân tạo (ANN): ANN là một mô hình học máy được lấy cảm hứng từ mạng lưới các tế bào thần kinh sinh học trong não bộ của chúng ta. Tuy nhiên, mặc dù máy bay được lấy cảm hứng từ chim, chúng không cần phải vỗ cánh. Tương tự, ANN dần trở nên khác biệt đáng kể so với các tế bào thần kinh sinh học. Một số nhà nghiên cứu thậm chí còn cho rằng chúng ta nên loại bỏ hoàn toàn sự tương đồng sinh học (ví dụ, bằng cách nói "các đơn vị" thay vì "các tế bào thần kinh"), để tránh hạn chế sự sáng tạo của chúng ta chỉ trong các hệ thống khả thi về mặt sinh học.

1

Mạng nơ-ron nhân tạo (ANN) là cốt lõi của Học sâu. Chúng rất linh hoạt, mạnh mẽ và có khả năng mở rộng, lý tưởng để giải quyết các tác vụ Học máy lớn và phức tạp như phân loại hàng tỷ hình ảnh (ví dụ: Google).

Hình ảnh), cung cấp năng lượng cho các dịch vụ nhận dạng giọng nói (ví dụ: Siri của Apple), đề xuất những video hay nhất để xem cho hàng trăm triệu người dùng mỗi ngày (ví dụ: YouTube), hoặc học cách đánh bại nhà vô địch thế giới trong trò chơi cờ vây (AlphaGo của DeepMind).

Phần đầu của chương này giới thiệu về mạng nơ-ron nhân tạo (ANN), bắt đầu bằng một cái nhìn tổng quan nhanh về các kiến trúc ANN đầu tiên và dẫn đến Mạng nơ-ron đa lớp (MLP), được sử dụng rộng rãi hiện nay (các kiến trúc khác sẽ được khám phá trong các chương tiếp theo). Trong phần thứ hai, chúng ta sẽ xem xét cách triển khai mạng nơ-ron bằng API Keras của TensorFlow. Đây là một API cấp cao được thiết kế đẹp mắt và đơn giản để xây dựng, huấn luyện, đánh giá và chạy mạng nơ-ron. Nhưng đừng để sự đơn giản của nó đánh lừa bạn: nó đủ mạnh mẽ và linh hoạt để cho phép bạn xây dựng nhiều kiến trúc mạng nơ-ron khác nhau. Trên thực tế, nó có thể đủ cho hầu hết các trường hợp sử dụng của bạn. Và nếu bạn cần thêm sự linh hoạt, bạn luôn có thể viết các thành phần Keras tùy chỉnh bằng cách sử dụng API cấp thấp hơn của nó, hoặc thậm chí sử dụng TensorFlow trực tiếp, như chúng ta sẽ thấy trong [Chương 12](#).

Nhưng trước tiên, hãy cùng quay ngược thời gian để xem mạng nơ-ron nhân tạo ra đời như thế nào!

Từ tế bào thần kinh sinh học đến tế bào thần kinh nhân tạo

Điều đáng ngạc nhiên là mạng nơ-ron nhân tạo (ANN) đã tồn tại khá lâu: chúng được giới thiệu lần đầu tiên vào năm 1943 bởi nhà thần kinh học Warren McCulloch và nhà toán học Walter Pitts. Trong [bài báo mang tính bước ngoặt](#) của họ Trong bài báo ² "Một phép tính logic về các ý tưởng tiềm ẩn trong hoạt động thần kinh", McCulloch và Pitts đã trình bày một mô hình tính toán đơn giản về cách các tế bào thần kinh sinh học có thể phối hợp với nhau trong não động vật để thực hiện các phép tính phức tạp bằng logic mệnh đề. Đây là kiến trúc mạng thần kinh nhân tạo đầu tiên. Kể từ đó, nhiều kiến trúc khác đã được phát minh, như chúng ta sẽ thấy.

Những thành công ban đầu của mạng nơ-ron nhân tạo (ANN) đã dẫn đến niềm tin rộng rãi rằng chúng ta sẽ sớm có thể trò chuyện với những cỗ máy thực sự thông minh. Khi rõ ràng vào những năm 1960 rằng lời hứa này sẽ không thành hiện thực (ít nhất là trong một thời gian khá dài),

Nguồn tài trợ chuyển hướng sang những lĩnh vực khác, và mạng nơ-ron nhân tạo (ANN) bước vào thời kỳ suy thoái kéo dài. Đầu những năm 1980, các kiến trúc mới được phát minh và các kỹ thuật huấn luyện tốt hơn được phát triển, khơi dậy sự quan tâm trở lại đối với thuyết kết nối, tức là nghiên cứu về mạng nơ-ron. Nhưng tiến độ chậm, và đến những năm 1990, các kỹ thuật Học máy mạnh mẽ khác được phát minh, chẳng hạn như Máy vectơ hỗ trợ (xem [Chương 5](#)). Những kỹ thuật này dường như mang lại kết quả tốt hơn và nền tảng lý thuyết vững chắc hơn so với ANN, vì vậy một lần nữa việc nghiên cứu về mạng nơ-ron lại bị tạm dừng.

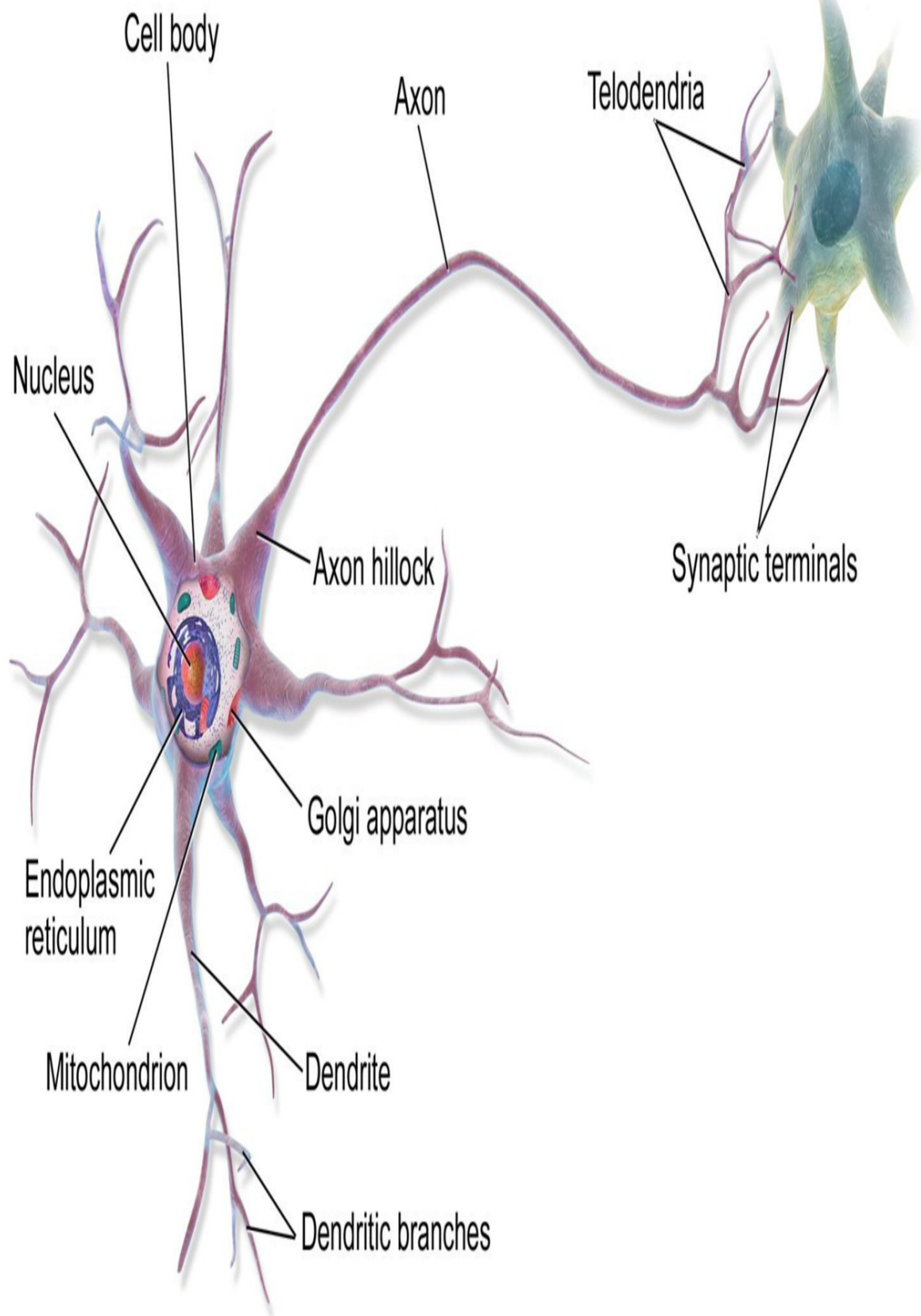
Chúng ta đang chứng kiến một làn sóng quan tâm mới đến mạng nơ-ron nhân tạo (ANN). Liệu làn sóng này sẽ lụi tàn như những làn sóng trước đây? Dưới đây là một vài lý do chính đáng để tin rằng làn sóng này sẽ khác và sự quan tâm mới đến ANN sẽ có tác động sâu sắc hơn nhiều đến cuộc sống của chúng ta:

- Hiện nay có một lượng dữ liệu khổng lồ sẵn có để huấn luyện mạng nơ-ron, và mạng nơ-ron nhân tạo (ANN) thường cho hiệu quả vượt trội so với các kỹ thuật học máy khác trên các bài toán rất lớn và phức tạp.
- Sự gia tăng mạnh mẽ về sức mạnh tính toán kể từ những năm 1990 đã giúp việc huấn luyện các mạng nơ-ron lớn trở nên khả thi trong một khoảng thời gian hợp lý. Điều này một phần là nhờ định luật Moore (số lượng linh kiện trong mạch tích hợp đã tăng gấp đôi sau khoảng 2 năm trong 50 năm qua), nhưng cũng nhờ vào ngành công nghiệp game, vốn đã thúc đẩy việc sản xuất hàng triệu card đồ họa GPU mạnh mẽ. Hơn nữa, các nền tảng điện toán đám mây đã giúp sức mạnh này trở nên dễ tiếp cận với mọi người.
- Các thuật toán huấn luyện đã được cải tiến. Công bằng mà nói, chúng chỉ khác một chút so với các thuật toán được sử dụng vào những năm 1990, nhưng những điều chỉnh tương đối nhỏ này đã mang lại tác động tích cực rất lớn.
- Một số hạn chế lý thuyết của mạng nơ-ron nhân tạo (ANN) hóa ra lại không gây ảnh hưởng gì trong thực tế. Ví dụ, nhiều người cho rằng các thuật toán huấn luyện ANN sẽ thất bại vì chúng dễ bị mắc kẹt ở các điểm tối ưu cục bộ, nhưng thực tế cho thấy điều này không phải là vấn đề lớn, đặc biệt đối với các mạng nơ-ron lớn hơn: các điểm tối ưu cục bộ thường hoạt động tốt gần như điểm tối ưu toàn cục.

- Mạng nơ-ron nhân tạo (ANN) dường như đã bước vào một vòng tuần hoàn tích cực về tài trợ và tiến bộ. Những sản phẩm tuyệt vời dựa trên mạng nơ-ron nhân tạo (ANN) thường xuyên xuất hiện trên các trang báo, thu hút ngày càng nhiều sự chú ý và nguồn vốn đầu tư, dẫn đến sự tiến bộ ngày càng tăng và tạo ra nhiều sản phẩm tuyệt vời hơn nữa.

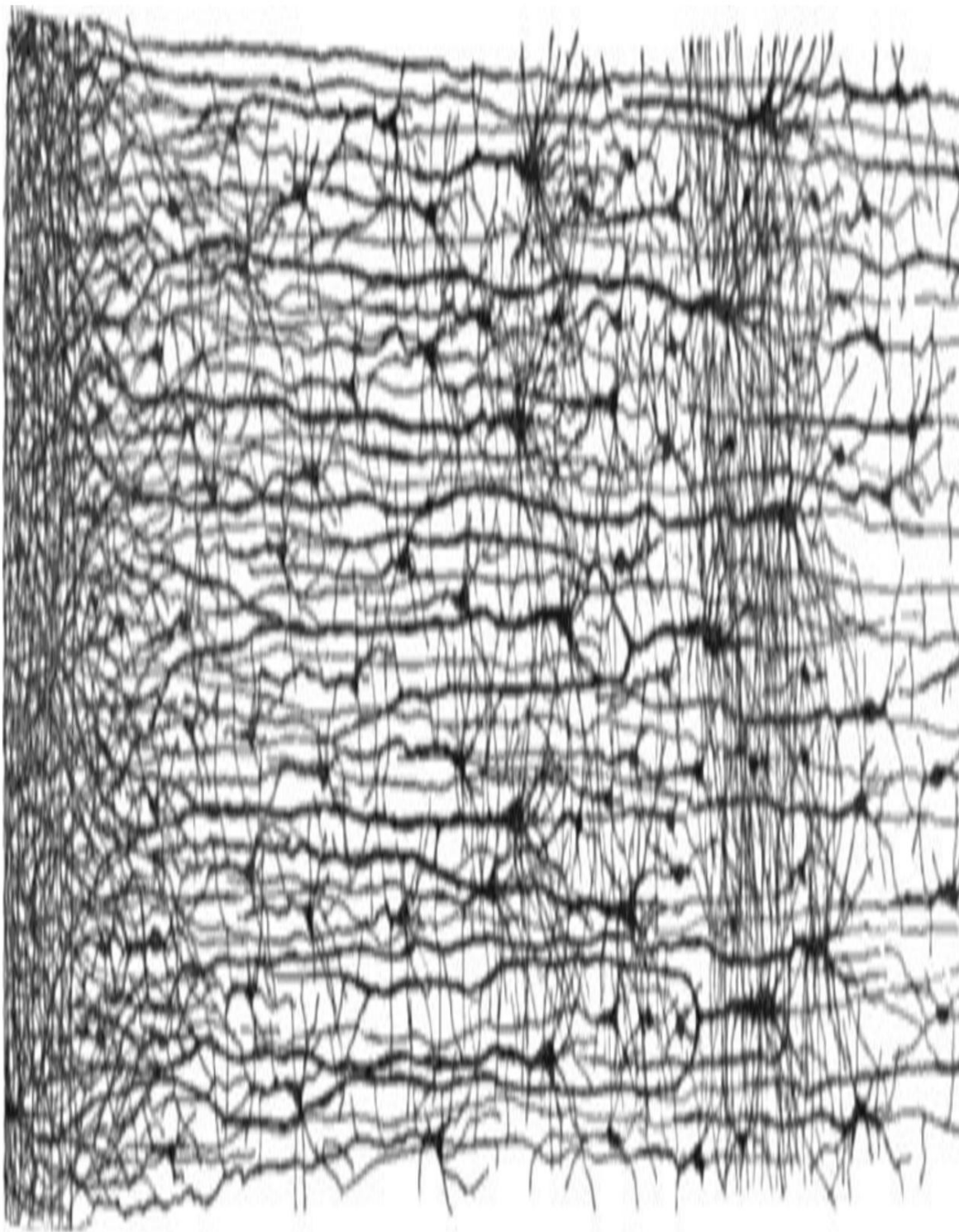
Các tế bào thần kinh sinh học

Trước khi thảo luận về các tế bào thần kinh nhân tạo, chúng ta hãy xem xét nhanh một tế bào thần kinh sinh học (được minh họa trong **Hình 10-1**). Đó là một tế bào có hình dạng khác thường, chủ yếu được tìm thấy trong não động vật. Nó bao gồm một thân tế bào chứa nhân và hầu hết các thành phần phức tạp của tế bào, nhiều phần mở rộng phân nhánh gọi là sợi nhánh (dendrites), cộng với một phần mở rộng rất dài gọi là sợi trục (axon). Chiều dài của sợi trục có thể chỉ dài hơn thân tế bào vài lần, hoặc dài hơn đến hàng chục nghìn lần. Gần đầu mút, sợi trục phân nhánh thành nhiều nhánh gọi là đầu mút nhánh (telodendria), và ở đầu các nhánh này là các cấu trúc nhỏ gọi là đầu cuối khớp thần kinh (synaptic terminals, hay đơn giản là synapse), được kết nối với các sợi nhánh hoặc thân tế bào của các tế bào thần kinh khác. Các tế bào thần kinh sinh học tạo ra các xung điện ngắn gọi là điện thế hoạt động (AP, hay chỉ là tín hiệu) truyền dọc theo sợi trục và làm cho các khớp thần kinh ³ giải phóng các tín hiệu hóa học gọi là chất dẫn truyền thần kinh. Khi một tế bào thần kinh nhận được đủ lượng chất dẫn truyền thần kinh này trong vòng vài mili giây, nó sẽ phát ra các xung điện của riêng mình (thực tế, điều này phụ thuộc vào các chất dẫn truyền thần kinh, vì một số chất trong số đó ức chế tế bào thần kinh phát xung).



Hình 10-1. Tế bào thần kinh sinh học

Như vậy, các tế bào thần kinh sinh học riêng lẻ dường như hoạt động theo một cách khá đơn giản, nhưng chúng được tổ chức trong một mạng lưới khổng lồ gồm hàng tỷ tế bào, với mỗi tế bào thần kinh thường được kết nối với hàng nghìn tế bào thần kinh khác. Các phép tính phức tạp cao có thể được thực hiện bởi một mạng lưới các tế bào thần kinh khá đơn giản, giống như một tổ kiến phức tạp có thể hình thành từ nỗ lực kết hợp của những con kiến đơn giản. Kiến trúc của mạng lưới thần kinh sinh học (BNN) vẫn đang là chủ đề nghiên cứu tích cực, nhưng một số phần của não đã được lập bản đồ, và dường như các tế bào thần kinh thường được tổ chức thành các lớp liên tiếp, đặc biệt là ở vỏ não (tức là lớp ngoài cùng của não), như được minh họa trong Hình 10-2.

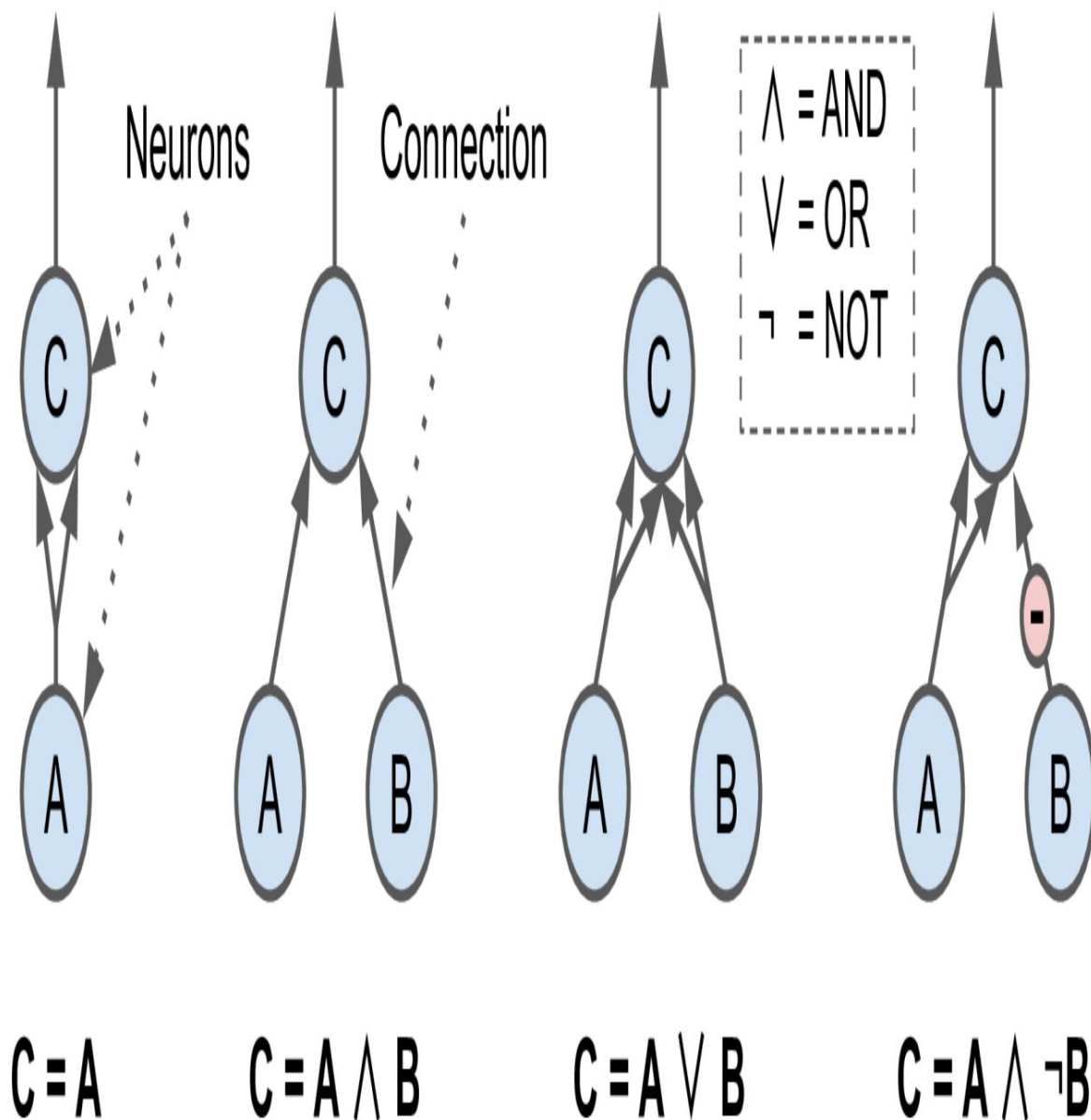


Hình 10-2. Nhiều lớp trong mạng lưới thần kinh sinh học (vỏ não người)6

Tính toán logic với nơ-ron

McCulloch và Pitts đã đề xuất một mô hình rất đơn giản của tế bào thần kinh sinh học, sau này được biết đến với tên gọi tế bào thần kinh nhân tạo: nó có một hoặc nhiều đầu vào nhị phân (bật/tắt) và một đầu ra nhị phân. Tế bào thần kinh nhân tạo kích hoạt đầu ra của nó khi có nhiều hơn một số lượng đầu vào nhất định hoạt động.

Trong bài báo của họ, các tác giả đã chỉ ra rằng ngay cả với mô hình đơn giản như vậy, vẫn có thể xây dựng một mạng lưới các nơ-ron nhân tạo tính toán bất kỳ mệnh đề logic nào bạn muốn. Để xem mạng lưới như vậy hoạt động như thế nào, chúng ta hãy xây dựng một vài mạng nơ-ron nhân tạo thực hiện các phép tính logic khác nhau (xem **Hình 10-3**), giả sử rằng một nơ-ron được kích hoạt khi ít nhất hai kết nối đầu vào của nó hoạt động.



Hình 10-3. Mạng nơ-ron nhân tạo thực hiện các phép tính logic đơn giản.

Hãy cùng xem các mạng lưới này làm gì:

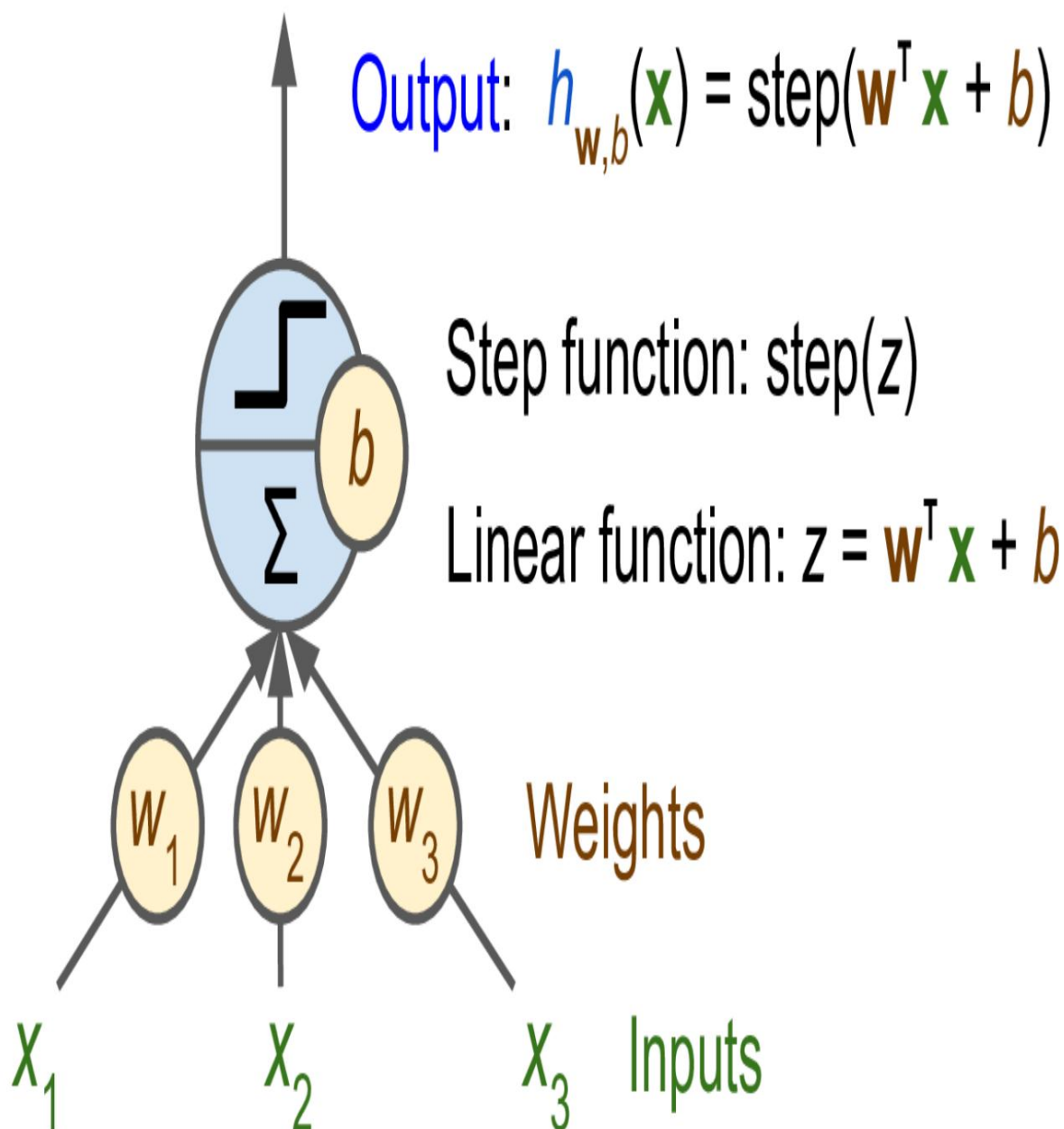
- Mạng lưới đầu tiên bên trái là hàm đồng nhất: nếu nơron A được kích hoạt, thì nơron C cũng được kích hoạt (vì nó nhận hai tín hiệu đầu vào từ nơron A); nhưng nếu nơron A tắt, thì nơron C cũng tắt.
- Mạng thứ hai thực hiện phép toán logic AND: nơron C chỉ được kích hoạt khi cả hai nơron A và B đều được kích hoạt (một tín hiệu đầu vào đơn lẻ không đủ để kích hoạt nơron C).

- Mạng lưới thứ ba thực hiện phép toán logic OR: nơ-ron C được kích hoạt nếu nơ-ron A hoặc nơ-ron B được kích hoạt (hoặc cả hai).
- Cuối cùng, nếu ta giả sử rằng một kết nối đầu vào có thể ức chế hoạt động của nơ-ron (điều này đúng với các nơ-ron sinh học), thì mạng thứ tư sẽ tính toán một mệnh đề logic phức tạp hơn một chút: nơ-ron C chỉ được kích hoạt khi nơ-ron A hoạt động và nơ-ron B tắt. Nếu nơ-ron A hoạt động mọi lúc, thì ta có một phép phủ định logic: nơ-ron C hoạt động khi nơ-ron B tắt, và ngược lại.

Bạn có thể hình dung cách các mạng này có thể được kết hợp để tính toán các biểu thức logic phức tạp (xem các bài tập ở cuối chương để biết ví dụ).

Perceptron

Mạng nơ-ron nhân tạo (ANN) là một trong những kiến trúc ANN đơn giản nhất, được Frank Rosenblatt phát minh vào năm 1957. Nó dựa trên một loại nơ-ron nhân tạo hơi khác (xem **Hình 10-4**) được gọi là đơn vị logic ngưỡng (TLU), hoặc đôi khi là đơn vị ngưỡng tuyến tính (LTU). Đầu vào và đầu ra là các số (thay vì giá trị nhị phân bật/tắt), và mỗi kết nối đầu vào được liên kết với một trọng số. TLU trước tiên tính toán một hàm tuyến tính của đầu vào: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w} \cdot \mathbf{x} + b$. Sau đó, nó áp dụng một hàm bậc thang cho kết quả: $h(\mathbf{x}) = \text{step}(z)$. Vì vậy, nó gần giống như Hồi quy Logistic, ngoại trừ việc nó sử dụng hàm bậc thang thay vì hàm logistic (**Chương 4**). Cũng giống như trong Hồi quy Logistic, các tham số của mô hình là các trọng số đầu vào w và số hạng thiên vị b .



Hình 10-4. Đơn vị logic ngưỡng: một nơon nhân tạo tính tổng có trọng số của các đầu vào $w x$, cộng với một số hạng thiên vị b , sau đó áp dụng một hàm bậc thang.

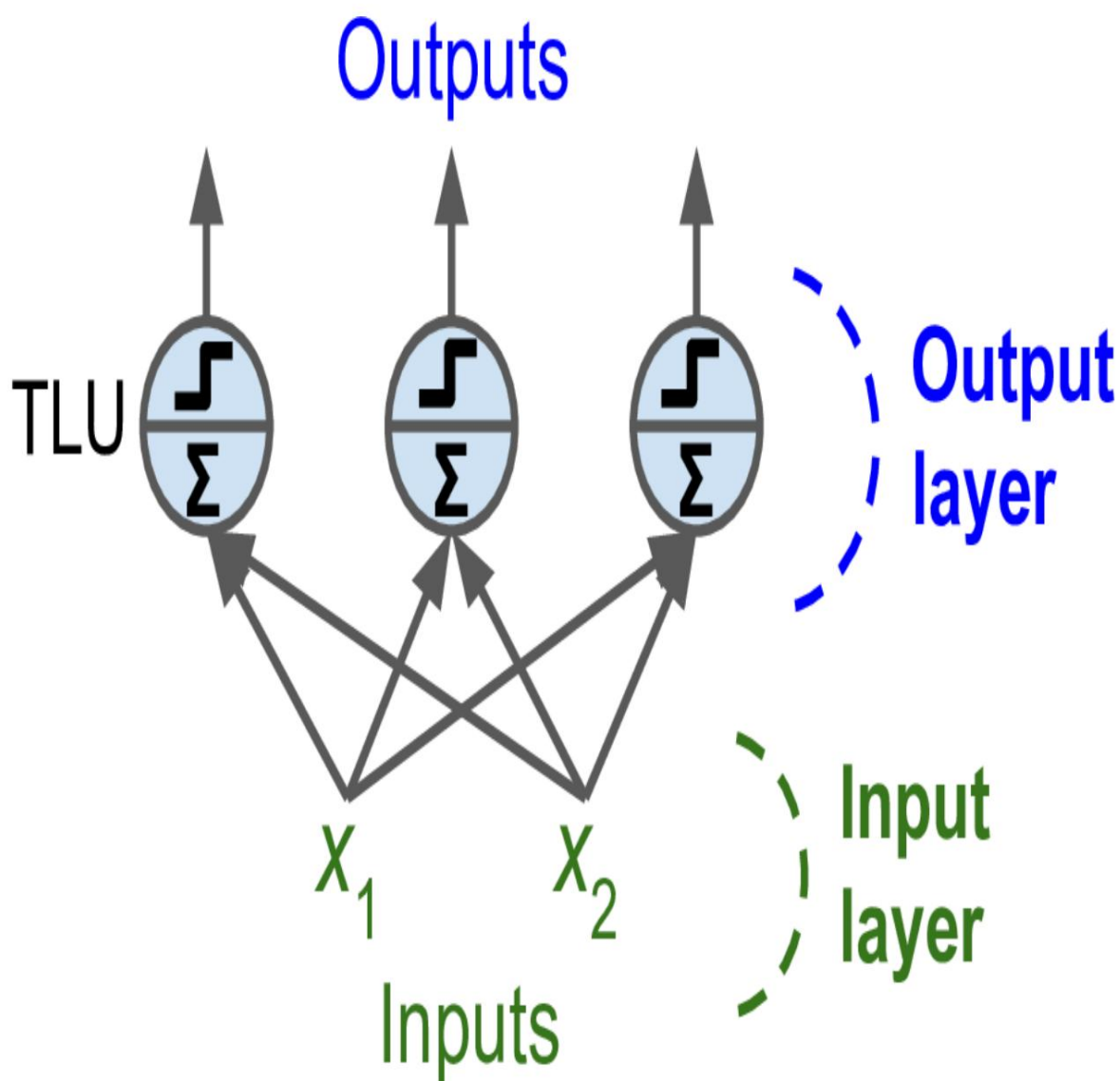
Hàm bậc thang phổ biến nhất được sử dụng trong Perceptron là hàm bậc thang Heaviside (xem [Phương trình 10-1](#)). Đôi khi hàm dấu được sử dụng thay thế.

Phương trình 10-1. Các hàm bậc thang thông dụng được sử dụng trong Perceptron (giả sử ngưỡng = 0)

$$\text{heaviside}(z) = \begin{cases} 0 & \text{nếu } z < 0 \\ 1 & \text{nếu } z \geq 0 \end{cases} \quad \text{sgn}(z) = \begin{cases} -1 & \text{nếu } z < 0 \\ 0 & \text{nếu } z = 0 \\ +1 & \text{nếu } z > 0 \end{cases}$$

Một TLU đơn lẻ có thể được sử dụng cho phân loại nhị phân tuyến tính đơn giản. Nó tính toán một hàm tuyến tính dựa trên các đầu vào của nó, và nếu kết quả vượt quá ngưỡng, nó sẽ xuất ra lớp tích cực. Nếu không, nó sẽ xuất ra lớp tiêu cực. Điều này có thể gợi nhớ đến Hồi quy Logistic (Chương 4) hoặc phân loại SVM tuyến tính (Chương 5). Ví dụ, bạn có thể sử dụng một TLU đơn lẻ để phân loại hoa diên vĩ dựa trên chiều dài và chiều rộng cánh hoa. Việc huấn luyện một TLU như vậy sẽ yêu cầu tìm ra các giá trị phù hợp cho w_1 , w_2 và b (việc đào tạo) w (thuật toán sẽ được thảo luận ngay sau đây).

Một Perceptron được cấu tạo từ một hoặc nhiều TLU được sắp xếp trong một lớp duy nhất, trong đó mỗi TLU được kết nối với mọi đầu vào. Lớp như vậy được gọi là lớp kết nối đầy đủ, hoặc lớp dày đặc. Các đầu vào tạo thành lớp đầu vào. Và vì lớp các TLU tạo ra các đầu ra cuối cùng, nên nó được gọi là lớp đầu ra. Ví dụ, một Perceptron với hai đầu vào và ba đầu ra được thể hiện trong Hình 10-5.



Hình 10-5. Kiến trúc của một Perceptron với hai đầu vào và ba nơ-ron đầu ra.

Perceptron này có thể phân loại các đối tượng đồng thời thành ba lớp nhị phân khác nhau, do đó nó là một bộ phân loại đa nhãn. Nó cũng có thể được sử dụng cho phân loại đa lớp.

Nhờ vào sức mạnh của đại số tuyến tính, **phương trình 10-2** có thể được sử dụng để tính toán hiệu quả đầu ra của một lớp nơ-ron nhân tạo cho nhiều trường hợp cùng một lúc.

Phương trình 10-2. Tính toán đầu ra của một lớp kết nối đầy đủ

$$h_{W,b}(X) = \varphi(XW + b)$$

Trong phương trình này:

- Như thường lệ, X biểu thị ma trận các đặc trưng đầu vào. Nó có một hàng cho mỗi trường hợp và một cột cho mỗi đặc trưng.
- Ma trận trọng số W chứa tất cả các trọng số kết nối. Nó có một hàng cho mỗi đầu vào và một cột cho mỗi nơ-ron.
- Vectơ độ lệch b chứa tất cả các số hạng độ lệch: một số hạng cho mỗi nơ-ron.
- Hàm φ được gọi là hàm kích hoạt: khi các nơ-ron nhân tạo là TLU, nó là một hàm bậc thang (nhưng chúng ta sẽ thảo luận về các hàm kích hoạt khác sau).

GHI CHÚ

Trong toán học, tổng của một ma trận và một vectơ là không xác định. Tuy nhiên, trong Khoa học Dữ liệu, chúng ta cho phép "phát sóng": cộng một vectơ vào một ma trận có nghĩa là cộng vectơ đó vào mọi hàng trong ma trận. Vì vậy, $XW + b$ trước tiên nhân X với W —kết quả là một ma trận với một hàng cho mỗi trường hợp và một cột cho mỗi đầu ra—sau đó cộng vectơ b vào mọi hàng của ma trận đó, điều này cộng mỗi số hạng thiên vị vào đầu ra tương ứng, cho mỗi trường hợp. Hơn nữa, φ được áp dụng riêng lẻ cho từng mục trong ma trận kết quả.

Vậy, Perceptron được huấn luyện như thế nào? Thuật toán huấn luyện Perceptron do Rosenblatt đề xuất phần lớn được lấy cảm hứng từ quy tắc Hebb. Trong cuốn sách "Tổ chức hành vi" (Wiley) năm 1949, Donald Hebb cho rằng khi một nơ-ron sinh học thường xuyên kích hoạt một nơ-ron khác, kết nối giữa hai nơ-ron này sẽ trở nên mạnh hơn. Siegrid Löwel sau đó đã tóm tắt ý tưởng của Hebb bằng câu nói dễ nhớ, "Các tế bào cùng kích hoạt sẽ cùng kết nối"; nghĩa là, trọng số kết nối giữa hai nơ-ron có xu hướng tăng lên khi chúng kích hoạt đồng thời. Quy tắc này sau đó được gọi là quy tắc Hebb (hoặc học Hebbian). Perceptron được huấn luyện bằng cách sử dụng một biến thể của quy tắc này, có tính đến lỗi mà mạng mắc phải khi đưa ra dự đoán; quy tắc học Perceptron củng cố các kết nối giúp giảm lỗi. Cụ thể hơn, Perceptron được cung cấp một trường hợp huấn luyện tại một thời điểm, và với mỗi trường hợp, nó đưa ra dự đoán của mình. Với mỗi

Nơ-ron đầu ra tạo ra dự đoán sai sẽ củng cố trọng số kết nối từ các đầu vào lẽ ra đã góp phần tạo ra dự đoán chính xác. Quy tắc này được thể hiện trong **Phương trình 10-3**.

Phương trình 10-3. Quy tắc học tập của perceptron (cập nhật trọng số)

$$\text{(bước tiếp theo) } w_{i,j} = w_{i,j} + \eta (y_j - \hat{y}_j) x_i$$

Trong phương trình này:

- $w_{i,j}$ là trọng số kết nối giữa đầu vào thứ i và nơ-ronth thứ j .
- x_i là giá trịth đầu vào thứ i của trường hợp huấn luyện hiện tại.
- $\hat{y}_j$ là đầu ra của nơ-ron đầu rath thứ j cho trường hợp huấn luyện hiện tại.
- y_j là đầu ra mục tiêu của nơ-ron đầu rath thứ j cho trường hợp huấn luyện hiện tại.
- η là tốc độ học (xem **Chương 4**).

Ranh giới quyết định của mỗi nơ-ron đầu ra là tuyến tính, do đó Perceptron không có khả năng học các mẫu phức tạp (giống như các bộ phân loại Hồi quy Logistic).

Tuy nhiên, nếu các trường hợp huấn luyện có thể phân tách tuyến tính, Rosenblatt đã chứng minh rằng thuật toán này sẽ hội tụ đến một giải pháp. Đây được gọi là định lý hội tụ Perceptron.

Scikit-Learn cung cấp lớp Perceptron, có thể được sử dụng khá giống như bạn mong đợi – ví dụ, trên tập dữ liệu iris (được giới thiệu trong...)

Chương 4):

```
import numpy as np from
sklearn.datasets import load_iris from
sklearn.linear_model import Perceptron

iris = load_iris(as_frame=True)
X = iris.data[["chiều dài cánh hoa (cm)", "chiều rộng cánh hoa (cm)"].values y =
(iris.target == 0) # Iris setosa

per_clf = Perceptron(random_state=42)
```

```
per_clf.fit(X, y)

X_new = [[2, 0.5], [3, 1]] y_pred
= per_clf.predict(X_new) # dự đoán True và False cho 2 bông hoa này
```

Có thể bạn đã nhận thấy rằng thuật toán học Perceptron rất giống với thuật toán Giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent - SGD) (đã được giới thiệu trong **Chương 4**). Trên thực tế, lớp Perceptron của Scikit-Learn tương đương với việc sử dụng SGDClassifier với các siêu tham số sau: `loss="perceptron"`, `learning_rate="constant"`, `eta0=1` (tốc độ học) và `penalty=None` (không điều chỉnh).

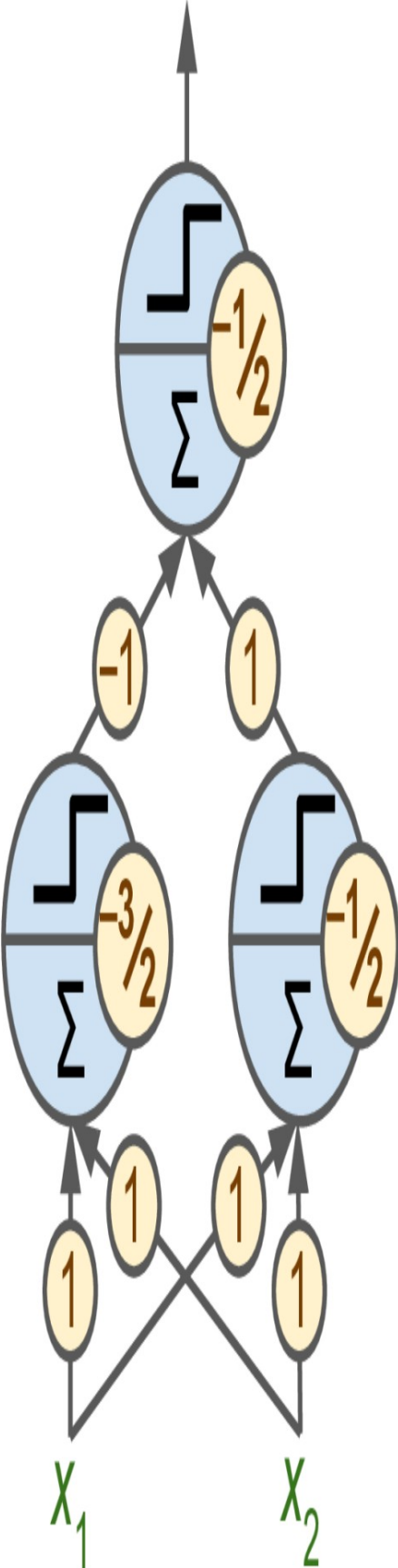
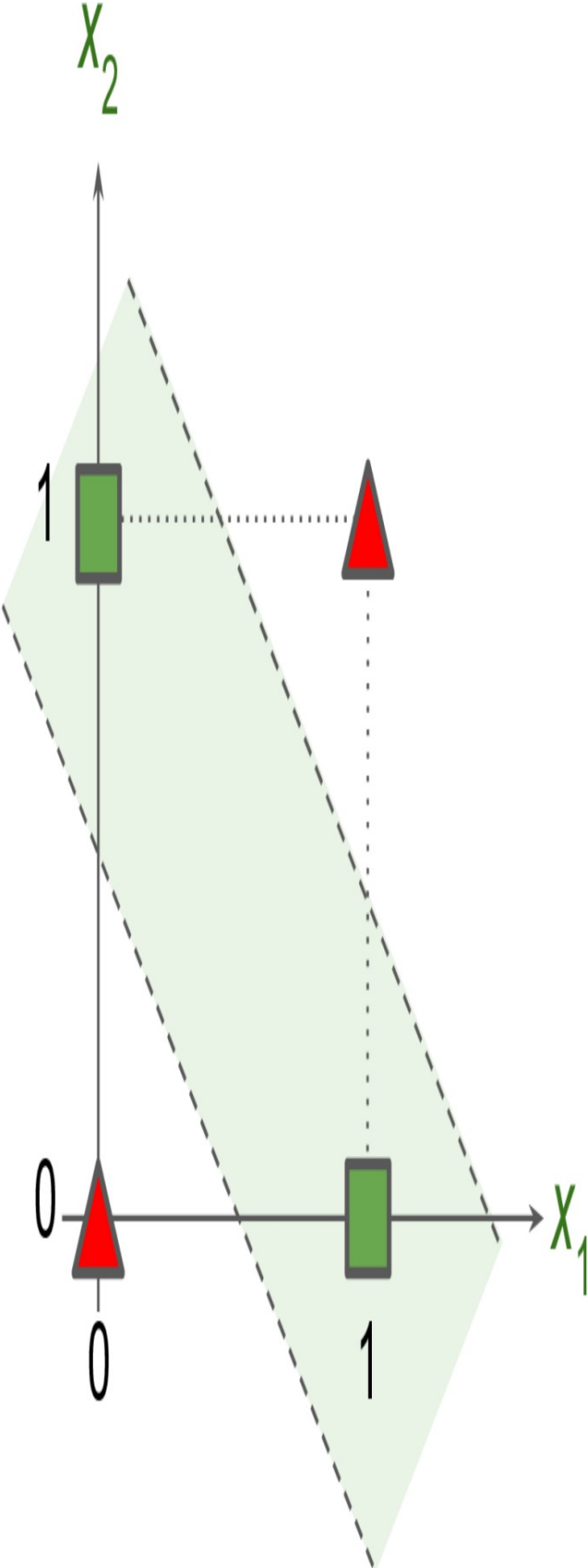
GHI CHÚ

Trái ngược với các thuật toán phân loại hồi quy logistic, mạng Perceptron không đưa ra xác suất cho từng lớp. Đây là một trong những lý do để ưu tiên hồi quy logistic hơn mạng Perceptron. Hơn nữa, Perceptron không sử dụng bất kỳ phương pháp điều chỉnh nào theo mặc định, và quá trình huấn luyện dừng lại ngay khi không còn lỗi dự đoán nào trên tập dữ liệu huấn luyện, do đó mô hình thường không tổng quát hóa tốt như Hồi quy Logistic hoặc bộ phân loại SVM tuyến tính. Tuy nhiên, Perceptron có thể huấn luyện nhanh hơn một chút.

Trong chuyên khảo Perceptrons năm 1969, Marvin Minsky và Seymour Papert đã chỉ ra một số điểm yếu nghiêm trọng của Perceptron—đặc biệt là việc chúng không có khả năng giải quyết một số vấn đề đơn giản (ví dụ: bài toán phân loại XOR (Exclusive OR); xem phía bên trái của **Hình 10-6**). Điều này đúng với bất kỳ mô hình phân loại tuyến tính nào khác (chẳng hạn như bộ phân loại Hồi quy Logistic), nhưng các nhà nghiên cứu đã kỳ vọng nhiều hơn từ Perceptron, và một số người đã thất vọng đến mức họ hoàn toàn từ bỏ mạng nơ-ron để chuyển sang các vấn đề cấp cao hơn như logic, giải quyết vấn đề và tìm kiếm. Việc thiếu các ứng dụng thực tiễn cũng không giúp ích gì.

Hóa ra, một số hạn chế của Perceptron có thể được khắc phục bằng cách xếp chồng nhiều Perceptron lên nhau. Mạng nơ-ron nhân tạo (ANN) thu được được gọi là Perceptron đa lớp (MLP). MLP có thể giải quyết bài toán XOR, như bạn có thể kiểm chứng.

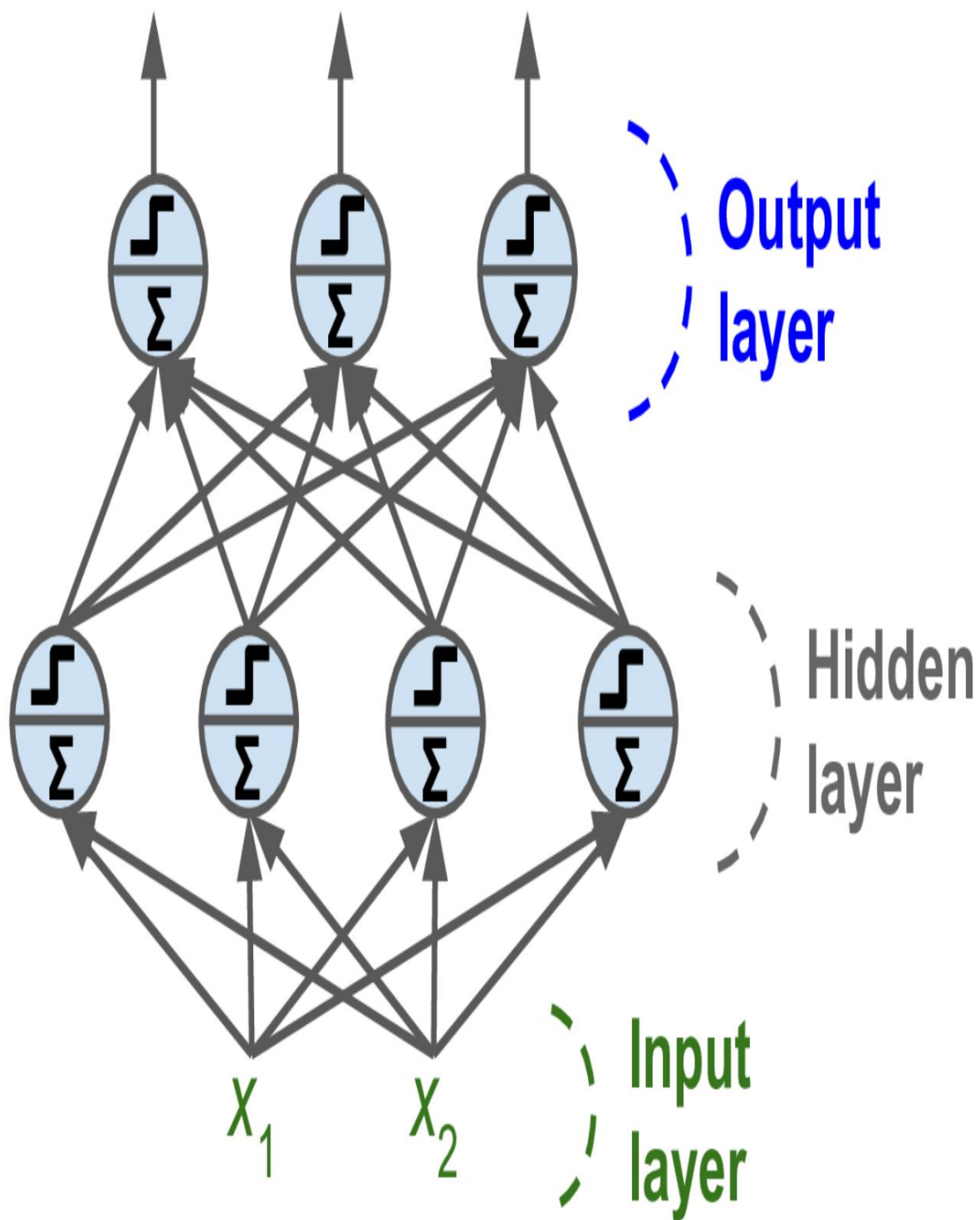
Bằng cách tính toán đầu ra của mạng MLP được biểu diễn ở phía bên phải của **Hình 10-6**: với đầu vào $(0, 0)$ hoặc $(1, 1)$, mạng sẽ xuất ra 0, và với đầu vào $(0, 1)$ hoặc $(1, 0)$ nó sẽ xuất ra 1. Hãy thử xác minh xem mạng này có thực sự giải quyết được bài toán XOR hay không!



Hình 10-6. Bài toán phân loại XOR và một mạng MLP giải quyết bài toán đó.

Perceptron đa lớp và lan truyền ngược

Một mạng nơ-ron đa lớp (MLP) bao gồm một lớp đầu vào, một hoặc nhiều lớp TLU (Transparent Lumenal Unit), được gọi là các lớp ẩn, và một lớp TLU cuối cùng được gọi là lớp đầu ra (xem **Hình 10-7**). Các lớp gần lớp đầu vào thường được gọi là các lớp dưới, và các lớp gần đầu ra thường được gọi là các lớp trên.



Hình 10-7. Kiến trúc của một mạng nơ-ron đa lớp (Multilayer Perceptron) với hai đầu vào, một lớp ẩn gồm bốn nơ-ron và ba nơ-ron đầu ra.

GHI CHÚ

Tín hiệu chỉ truyền theo một chiều (từ đầu vào đến đầu ra), do đó kiến trúc này là một ví dụ về mạng nơ-ron truyền thẳng (FNN).

Khi một mạng nơ-ron nhân tạo (ANN) chứa một chồng lớp ẩn dày, nó được gọi là mạng nơ-ron sâu (DNN). Lĩnh vực Học sâu nghiên cứu về DNN, và nói chung, nó quan tâm đến các mô hình chứa nhiều lớp tính toán dày. Mặc dù vậy, nhiều người vẫn nói về Học sâu bất cứ khi nào có liên quan đến mạng nơ-ron (ngay cả những mạng có lớp ẩn nông).

Trong nhiều năm, các nhà nghiên cứu đã nỗ lực tìm cách huấn luyện mạng nơ-ron truyền đa lớp (MLP) mà không thành công. Đầu những năm 1960, một số nhà nghiên cứu đã thảo luận về khả năng sử dụng thuật toán Gradient Descent để huấn luyện mạng nơ-ron truyền, nhưng như chúng ta đã thấy trong **Chương 4**, Gradient Descent yêu cầu tính toán độ dốc của sai số mô hình đối với các tham số của mô hình: vào thời điểm đó, người ta chưa rõ làm thế nào để thực hiện điều này một cách hiệu quả với một mô hình phức tạp chứa nhiều tham số như vậy, đặc biệt là với những máy tính mà họ có vào thời điểm đó.

Vào năm 1970, một nhà nghiên cứu tên là Seppo Linnainmaa đã giới thiệu trong luận văn thạc sĩ của mình một kỹ thuật để tính toán tất cả các đạo hàm một cách tự động và hiệu quả. Thuật toán này hiện được gọi là phương pháp vi phân tự động ngược (hoặc gọi tắt là phương pháp vi phân tự động ngược). Chỉ với hai lần chạy qua mạng (một lần tiến, một lần lùi), nó có thể tính toán đạo hàm của lỗi mạng nơ-ron đối với từng tham số của mô hình. Nói cách khác, nó có thể tìm ra cách điều chỉnh trọng số của mỗi kết nối và độ lệch để giảm lỗi của mạng nơ-ron.

Các đạo hàm này sau đó có thể được sử dụng để thực hiện bước Giảm độ dốc (Gradient Descent). Nếu bạn lặp lại quá trình tính toán đạo hàm tự động và thực hiện bước Giảm độ dốc này, lỗi của mạng nơ-ron sẽ giảm dần cho đến khi đạt mức tối thiểu. Sự kết hợp giữa phương pháp tự động tính đạo hàm ngược và Giảm độ dốc này được gọi là lan truyền ngược (hoặc backprop).

GHI CHÚ

Có nhiều kỹ thuật tự động tính vi phân khác nhau, mỗi kỹ thuật đều có ưu điểm và nhược điểm riêng. Tự động tính vi phân theo chế độ đảo ngược rất phù hợp khi hàm cần tính vi phân có nhiều biến (ví dụ: trọng số kết nối và độ lệch) và ít đầu ra (ví dụ: một hàm mất mát). Nếu bạn muốn tìm hiểu thêm về tự động tính vi phân, hãy xem [Phụ lục B](#).

Trên thực tế, thuật toán lan truyền ngược có thể được áp dụng cho tất cả các loại đồ thị tính toán, không chỉ mạng nơ-ron: thực tế, luận văn thạc sĩ của Linnaïma không phải về mạng nơ-ron, mà là về lĩnh vực tổng quát hơn. Phải mất thêm vài năm nữa trước khi thuật toán lan truyền ngược bắt đầu được sử dụng để huấn luyện mạng nơ-ron, nhưng nó vẫn chưa phổ biến. Sau đó, David Rumelhart đã tái phát minh thuật toán lan truyền ngược (cũng như một số nhà nghiên cứu khác trong các lĩnh vực khác), và ông đã xuất bản một [bài báo mang tính đột phá](#)¹⁰. Năm 1986, cùng với Geoffrey Hinton và Ronald Williams, họ đã phân tích cách lan truyền ngược cho phép mạng nơ-ron học được các biểu diễn nội bộ hữu ích. Kết quả của họ ấn tượng đến mức lan truyền ngược nhanh chóng trở nên phổ biến trong lĩnh vực này. Ngày nay, nó là kỹ thuật huấn luyện phổ biến nhất cho mạng nơ-ron.

Vậy chúng ta hãy cùng xem lại thuật toán lan truyền ngược một cách chi tiết hơn:

- Nó xử lý từng lô nhỏ một (ví dụ: mỗi lô chứa 32 trường hợp) và duyệt qua toàn bộ tập dữ liệu huấn luyện nhiều lần. Mỗi lượt xử lý được gọi là một kỷ nguyên.
- Mỗi mini-batch đi vào mạng thông qua lớp đầu vào. Thuật toán sau đó tính toán đầu ra của tất cả các nơ-ron trong lớp ẩn đầu tiên, cho mỗi trường hợp trong mini-batch. Kết quả được truyền đến lớp tiếp theo, đầu ra của nó được tính toán và truyền đến lớp tiếp theo nữa, và cứ thế tiếp tục cho đến khi ta nhận được đầu ra của lớp cuối cùng, lớp đầu ra. Đây là quá trình truyền tiến: nó hoàn toàn giống như việc đưa ra dự đoán, ngoại trừ tất cả các kết quả trung gian đều được lưu giữ vì chúng cần thiết cho quá trình truyền ngược.
- Tiếp theo, thuật toán đo lường lỗi đầu ra của mạng (tức là, nó sử dụng một hàm mất mát để so sánh đầu ra mong muốn và đầu ra thực tế).

mạng lưới và trả về một số thước đo về lỗi).

- Tiếp theo, nó tính toán mức độ đóng góp của từng độ lệch đầu ra và từng kết nối đến lớp đầu ra vào lỗi. Việc này được thực hiện một cách phân tích bằng cách áp dụng quy tắc chuỗi (có lẽ là quy tắc cơ bản nhất trong giải tích), giúp bước này diễn ra nhanh chóng và chính xác.
- Thuật toán sau đó đo lường mức độ đóng góp lỗi đến từ mỗi kết nối trong lớp bên dưới, một lần nữa sử dụng quy tắc chuỗi, hoạt động ngược trở lại cho đến khi thuật toán đạt đến lớp đầu vào. Như đã giải thích trước đó, quá trình đảo ngược này đo lường hiệu quả độ dốc lỗi trên tất cả các trọng số và độ lệch kết nối trong mạng bằng cách truyền độ dốc lỗi ngược trở lại qua mạng (do đó có tên gọi là thuật toán).
- Cuối cùng, thuật toán thực hiện bước Giảm độ dốc (Gradient Descent) để điều chỉnh tất cả các trọng số kết nối trong mạng, sử dụng độ dốc lỗi mà nó vừa tính toán được.

Tóm lại, thuật toán lan truyền ngược đưa ra dự đoán cho một mini-batch (lượt truyền tiến), đo lỗi, sau đó đi ngược lại qua từng lớp để đo đóng góp lỗi từ mỗi tham số (lượt truyền ngược), và cuối cùng điều chỉnh trọng số và độ lệch kết nối để giảm lỗi (bước Gradient Descent).

CẢNH BÁO

Điều quan trọng là phải khởi tạo ngẫu nhiên tất cả các trọng số kết nối của các lớp ẩn, nếu không quá trình huấn luyện sẽ thất bại. Ví dụ, nếu bạn khởi tạo tất cả các trọng số và độ lệch bằng 0, thì tất cả các nơon trong một lớp nhất định sẽ hoàn toàn giống hệt nhau, và do đó thuật toán lan truyền ngược sẽ tác động đến chúng theo cùng một cách, vì vậy chúng sẽ vẫn giống hệt nhau. Nói cách khác, mặc dù có hàng trăm nơon trên mỗi lớp, mô hình của bạn sẽ hoạt động như thể nó chỉ có một nơon trên mỗi lớp: nó sẽ không thông minh lắm. Nếu thay vào đó bạn khởi tạo ngẫu nhiên các trọng số, bạn sẽ phá vỡ tính đối xứng và cho phép thuật toán lan truyền ngược huấn luyện một nhóm nơon đa dạng.

Để thuật toán lan truyền ngược hoạt động đúng cách, Rumelhart và các đồng nghiệp đã thực hiện một thay đổi quan trọng đối với kiến trúc của MLP: họ thay thế hàm bậc thang bằng...

Hàm logistic, $\sigma(z) = 1 / (1 + \exp(-z))$, còn được gọi là hàm sigmoid. Điều này rất cần thiết vì hàm bậc thang chỉ chứa các đoạn phẳng, do đó không có độ dốc để làm việc (Thuật toán Gradient Descent không thể di chuyển trên bề mặt phẳng), trong khi hàm sigmoid có đạo hàm khác 0 được xác định rõ ràng ở mọi nơi, cho phép Gradient Descent tiến bộ ở mỗi bước. Trên thực tế, thuật toán lan truyền ngược hoạt động tốt với nhiều hàm kích hoạt khác, không chỉ hàm sigmoid. Dưới đây là hai lựa chọn phổ biến khác:

Hàm tang hyperbolic: $\tanh(z) = 2\sigma(2z) - 1$

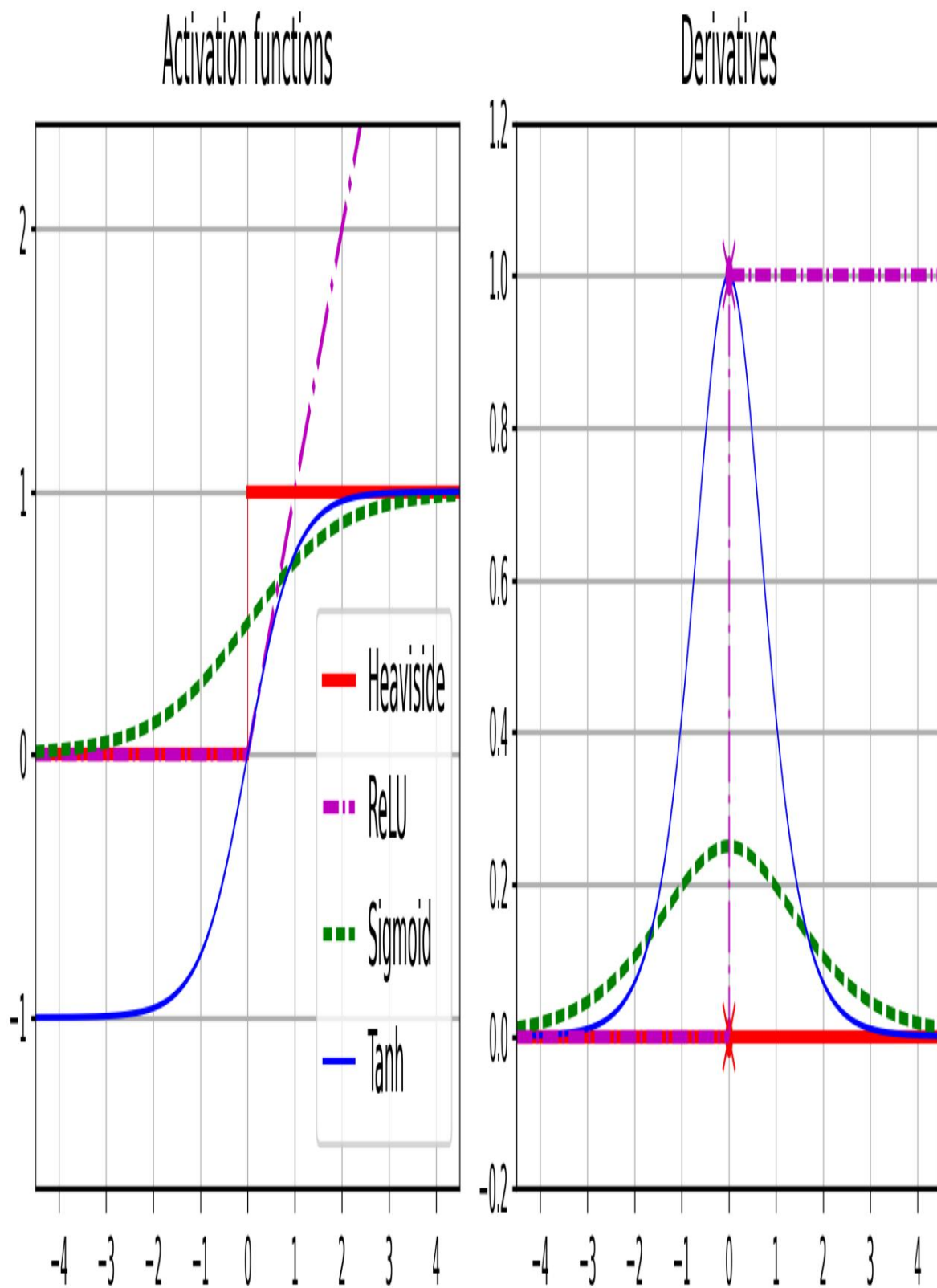
Giống như hàm sigmoid, hàm kích hoạt này có dạng chữ S, liên tục và khả vi, nhưng giá trị đầu ra của nó nằm trong khoảng từ -1 đến 1 (thay vì từ 0 đến 1 như trường hợp của hàm sigmoid). Khoảng giá trị đó có xu hướng làm cho đầu ra của mỗi lớp tập trung gần như xung quanh 0 ở giai đoạn đầu huấn luyện, điều này thường giúp tăng tốc độ hội tụ.

Hàm đơn vị tuyến tính hiệu chỉnh: $\text{ReLU}(z) = \max(0, z)$

Hàm ReLU liên tục nhưng không may là không khả vi tại $z = 0$ (độ dốc thay đổi đột ngột, điều này có thể khiến thuật toán Gradient Descent bị dao động), và đạo hàm của nó bằng 0 khi $z < 0$. Tuy nhiên, trên thực tế, nó hoạt động rất tốt và có ưu điểm là tính toán nhanh, vì vậy nó đã trở thành hàm mặc định. Quan trọng hơn, việc nó không¹¹ có giá trị đầu ra tối đa giúp giảm thiểu một số vấn đề trong quá trình Gradient Descent (chúng ta sẽ quay lại vấn đề này trong **Chương 11**).

Các hàm kích hoạt phổ biến này và các đạo hàm của chúng được thể hiện trong **Hình 10-8**. Nhưng khoan đã! Tại sao chúng ta lại cần các hàm kích hoạt ngay từ đầu? Chà, nếu bạn nối chuỗi nhiều phép biến đổi tuyến tính, tất cả những gì bạn nhận được chỉ là một phép biến đổi tuyến tính. Ví dụ, nếu $f(x) = 2x + 3$ và $g(x) = 5x - 1$, thì việc nối chuỗi hai hàm tuyến tính này sẽ cho bạn một hàm tuyến tính khác: $f(g(x)) = 2(5x - 1) + 3 = 10x + 1$. Vì vậy, nếu bạn không có một số phi tuyến tính giữa các lớp, thì ngay cả một chồng lớp sâu cũng tương đương với một lớp đơn, và bạn không thể giải quyết các vấn đề rất phức tạp với điều đó. Ngược lại, một

Về mặt lý thuyết, một mạng nơ-ron sâu (DNN) đủ lớn với các hàm kích hoạt phi tuyến tính có thể xấp xỉ bất kỳ hàm liên tục nào.



Hình 10-8. Hàm kích hoạt (bên trái) và đạo hàm của chúng (bên phải)

Được rồi! Bạn đã biết mạng nơ-ron xuất hiện từ đâu, kiến trúc của chúng như thế nào và cách tính toán đầu ra. Bạn cũng đã học về thuật toán lan truyền ngược. Nhưng chính xác thì bạn có thể làm gì với mạng nơ-ron?

Mạng nơ-ron truyền đa lớp hồi quy (Regression MLP)

Đầu tiên, MLP có thể được sử dụng cho các tác vụ hồi quy. Nếu bạn muốn dự đoán một giá trị duy nhất (ví dụ: giá nhà, dựa trên nhiều đặc điểm của nó), thì bạn chỉ cần một nơ-ron đầu ra duy nhất: đầu ra của nó là giá trị được dự đoán. Đối với hồi quy đa biến (tức là để dự đoán nhiều giá trị cùng một lúc), bạn cần một nơ-ron đầu ra cho mỗi chiều đầu ra. Ví dụ, để xác định vị trí trung tâm của một đối tượng trong hình ảnh, bạn cần dự đoán tọa độ 2D, vì vậy bạn cần hai nơ-ron đầu ra. Nếu bạn cũng muốn đặt một khung bao quanh đối tượng, thì bạn cần thêm hai số nữa: chiều rộng và chiều cao của đối tượng. Vì vậy, cuối cùng bạn sẽ cần bốn nơ-ron đầu ra.

Scikit-Learn bao gồm lớp `MLPRegressor`, vì vậy chúng ta hãy sử dụng nó để xây dựng một mạng nơ-ron đa lớp (MLP) với ba lớp ẩn, mỗi lớp gồm 50 nơ-ron, và huấn luyện nó trên tập dữ liệu nhà ở California. Để đơn giản, chúng ta sẽ sử dụng hàm `fetch_california_housing()` của Scikit-Learn để tải dữ liệu. Tập dữ liệu này đơn giản hơn tập dữ liệu chúng ta đã sử dụng trong [Chương 2](#), vì nó chỉ chứa các đặc trưng số (không có đặc trưng `ocean_proximity`), và không có giá trị thiếu. Đoạn mã sau bắt đầu bằng việc tải và chia tập dữ liệu, sau đó tạo một pipeline để chuẩn hóa các đặc trưng đầu vào trước khi gửi chúng đến `MLPRegressor`. Điều này rất quan trọng đối với mạng nơ-ron vì chúng được huấn luyện bằng thuật toán Gradient Descent, và như chúng ta đã thấy trong [Chương 4](#), Gradient Descent không hội tụ tốt khi các đặc trưng có thang đo rất khác nhau. Cuối cùng, đoạn mã huấn luyện mô hình và đánh giá lỗi xác thực của nó. Mô hình sử dụng hàm kích hoạt ReLU trong các lớp ẩn và sử dụng một biến thể của thuật toán Gradient Descent gọi là Adam (xem [Chương 11](#)) để giảm thiểu sai số bình phương trung bình, với một chút chuẩn hóa ℓ_2 (mà bạn có thể kiểm soát thông qua siêu tham số α).

```
from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
```

```

from sklearn.neural_network import MLPRegressor from sklearn.pipeline
import make_pipeline from sklearn.preprocessing import
StandardScaler

nhà ở = lấy_nhà_ở_california()
X_train_full, X_test, y_train_full, y_test = train_test_split( housing.data,
    housing.target, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split( X_train_full,
    y_train_full, random_state=42)

mlp_reg = MLPRegressor(hidden_layer_sizes=[50, 50, 50], random_state=42)
pipeline =
make_pipeline(StandardScaler(), mlp_reg) pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_valid) rmse =
mean_squared_error(y_valid, y_pred, squared=False)
# khoảng 0.505

```

Chúng ta thu được RMSE trên tập dữ liệu kiểm chứng khoảng 0,505, tương đương với kết quả thu được khi sử dụng thuật toán phân loại Rừng ngẫu nhiên. Không tệ chút nào cho lần thử đầu tiên!

Lưu ý rằng MLP này không sử dụng bất kỳ hàm kích hoạt nào cho lớp đầu ra, vì vậy nó có thể tự do xuất ra bất kỳ giá trị nào nó muốn. Điều này nhìn chung là ổn, nhưng nếu bạn muốn đảm bảo rằng đầu ra luôn dương, thì bạn nên sử dụng hàm kích hoạt ReLU trong lớp đầu ra, hoặc hàm kích hoạt softplus, là một biến thể mượt mà của ReLU: $\text{softplus}(z) = \log(1 + \exp(z))$.

Softplus gần bằng 0 khi z âm và gần bằng z khi z dương. Cuối cùng, nếu bạn muốn đảm bảo rằng các dự đoán luôn nằm trong một phạm vi giá trị nhất định, thì bạn nên sử dụng hàm sigmoid hoặc hàm hyperbolic tangent, và điều chỉnh tỷ lệ các mục tiêu đến phạm vi thích hợp: từ 0 đến 1 đối với sigmoid và từ -1 đến 1 đối với tanh. Đáng tiếc là lớp MLPRegressor không hỗ trợ các hàm kích hoạt trong lớp đầu ra.

CẢNH BÁO

Việc xây dựng và huấn luyện một mạng nơ-ron đa lớp (MLP) tiêu chuẩn bằng Scikit-Learn chỉ với vài dòng mã rất tiện lợi, nhưng các tính năng của mạng nơ-ron lại bị hạn chế. Đó là lý do tại sao chúng ta sẽ chuyển sang sử dụng Keras trong phần thứ hai của chương này.

Lớp `MLPRegressor` sử dụng sai số bình phương trung bình (MSE), thường là chỉ số bạn cần cho hồi quy, nhưng nếu tập dữ liệu huấn luyện có nhiều giá trị ngoại lệ, bạn có thể thích sử dụng sai số tuyệt đối trung bình (MAE) hơn. Hoặc, bạn có thể muốn sử dụng hàm mất mát Huber, là sự kết hợp của cả hai. Nó có dạng bậc hai khi sai số nhỏ hơn ngưỡng δ (thường là 1) nhưng có dạng tuyến tính khi sai số lớn hơn δ . Phần tuyến tính làm cho nó ít nhạy cảm với các giá trị ngoại lệ hơn so với sai số bình phương trung bình, và phần bậc hai cho phép nó hội tụ nhanh hơn và chính xác hơn so với MAE. Tuy nhiên, `MLPRegressor` chỉ hỗ trợ MSE.

Bảng 10-1 tóm tắt kiến trúc điển hình của một mạng nơ-ron đa lớp (MLP) hồi quy.

T

Một

bl

e

1

0- 1.

T

y pi

c

al

nốt rê

g

nốt rê

ss

io

N

M

L

P

Một

rc

CHÀO

te

ct

bạn

nốt rê

Siêu tham số Giá trị điển hình

các lớp ẩn

Tùy thuộc vào vấn đề, nhưng thông thường là từ 1 đến 5.

số nơ-ron trên mỗi lớp ẩn	Tùy thuộc vào vấn đề, nhưng thông thường là từ 10 đến 100.
# nơ-ron đầu ra	1 cho mỗi chiều dự đoán
Kích hoạt ẩn	ReLU
Kích hoạt đầu ra	Không dùng hàm nào, hoặc dùng ReLU/softplus (nếu đầu ra dương) hoặc sigmoid/tanh (nếu đầu ra bị chặn).
Hàm mất mát	MSE, hoặc Huber nếu có giá trị ngoại lệ.

Phân loại MLP

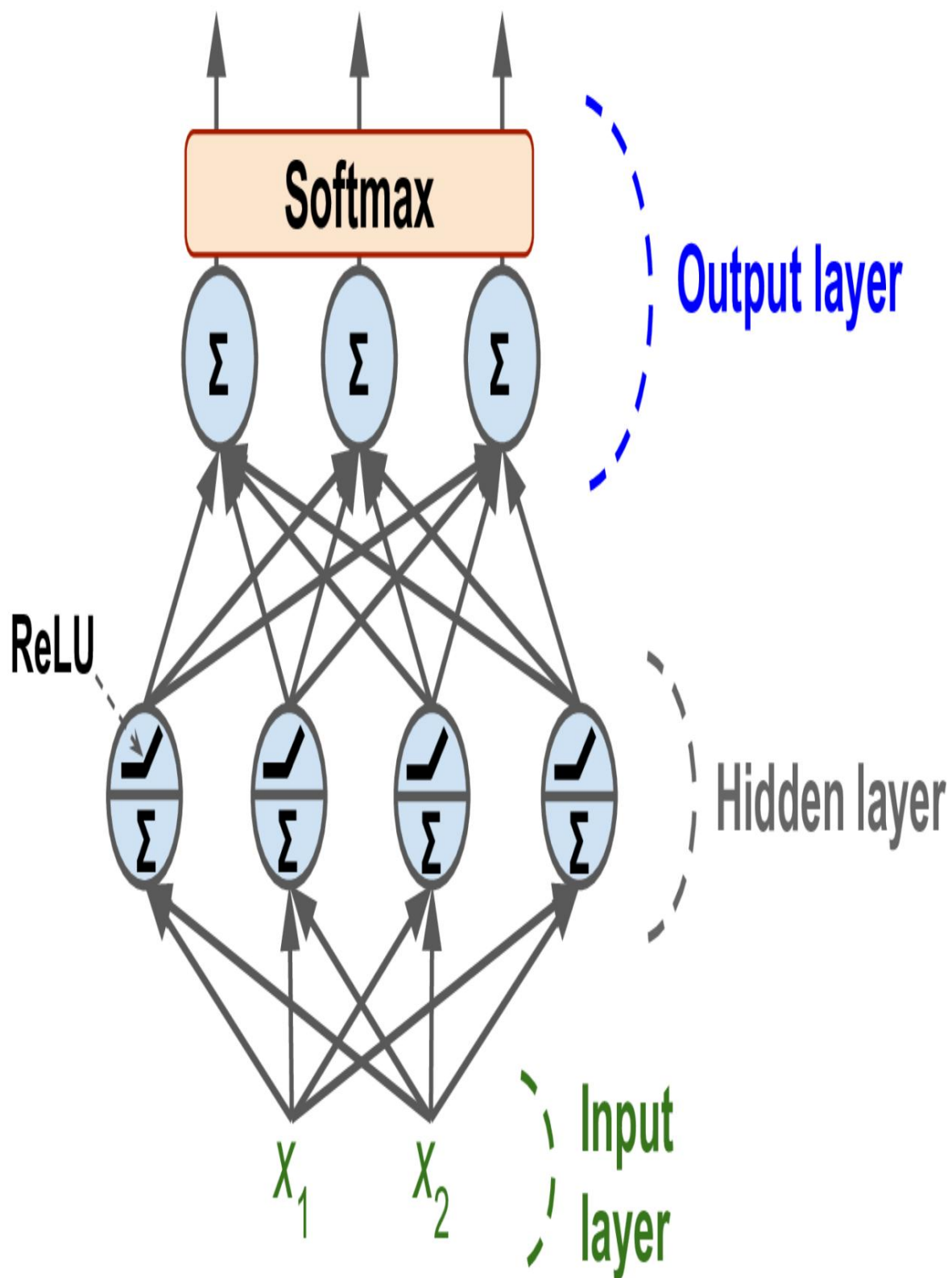
Mạng nơ-ron đa lớp (MLP) cũng có thể được sử dụng cho các bài toán phân loại. Đối với bài toán phân loại nhị phân, bạn chỉ cần một nơ-ron đầu ra duy nhất sử dụng hàm kích hoạt sigmoid: đầu ra sẽ là một số nằm giữa 0 và 1, mà bạn có thể hiểu là xác suất ước tính của lớp tích cực. Xác suất ước tính của lớp tiêu cực bằng một trừ đi số đó.

Mạng nơ-lan truyền đa lớp (MLP) cũng có thể dễ dàng xử lý các tác vụ phân loại nhị phân đa nhãn (xem [Chương 3](#)). Ví dụ, bạn có thể có một hệ thống phân loại email dự đoán xem mỗi email đến là thư hợp lệ hay thư rác, đồng thời dự đoán xem đó là email khẩn cấp hay không khẩn cấp. Trong trường hợp này, bạn cần hai nơ-ron đầu ra, cả hai đều sử dụng hàm kích hoạt sigmoid: nơ-ron đầu tiên sẽ xuất ra xác suất email là thư rác, và nơ-ron thứ hai sẽ xuất ra xác suất email đó là khẩn cấp. Tổng quát hơn, bạn sẽ dành một nơ-ron đầu ra cho mỗi lớp tích cực. Lưu ý rằng tổng xác suất đầu ra không nhất thiết phải bằng 1. Điều này cho phép mô hình xuất ra bất kỳ sự kết hợp nào của các nhãn: bạn có thể có thư hợp lệ không khẩn cấp, thư hợp lệ khẩn cấp, thư rác không khẩn cấp, và thậm chí có thể là thư rác khẩn cấp (mặc dù đó có thể là một lỗi).

Nếu mỗi trường hợp chỉ có thể thuộc về một lớp duy nhất, trong số ba lớp trở lên có thể có (ví dụ: các lớp từ 0 đến 9 cho phân loại ảnh chữ số), thì bạn cần có một nơ-ron đầu ra cho mỗi lớp, và bạn nên sử dụng...

Hàm kích hoạt softmax cho toàn bộ lớp đầu ra (xem [Hình 10-9](#)).

Hàm softmax (được giới thiệu trong [Chương 4](#)) sẽ đảm bảo rằng tất cả các xác suất ước tính đều nằm giữa 0 và 1, và tổng của chúng bằng 1 vì các lớp là độc lập với nhau. Điều này được gọi là phân loại đa lớp.



Hình 10-9. Một mạng MLP hiện đại (bao gồm ReLU và softmax) phân loại

Về hàm mất mát, vì chúng ta đang dự đoán các phân bố xác suất, nên hàm mất mát entropy chéo (hay còn gọi là x-entropy hoặc log loss, xem **Chương 4**) thường là một lựa chọn tốt.

Scikit-Learn có lớp `MLPClassifier` trong gói

`sklearn.neural_network`. Nó gần như giống hệt lớp `MLPRegressor`, ngoại trừ việc nó tối thiểu hóa entropy chéo thay vì MSE. Hãy thử ngay bây giờ, ví dụ trên tập dữ liệu Iris. Đây gần như là một bài toán tuyến tính, vì vậy một lớp đơn với 5 đến 10 nơon là đủ (và nhớ chuẩn hóa các đặc trưng).

Bảng 10-2 tóm tắt kiến trúc điển hình của một mạng MLP phân loại.

T

Một

bl

e

1 0-

2.

T

y pi

c

al

cl

Một

ss

ifi

c

Tại

io

N

M

L

P

Một

rc

CHÀO

te

ct

bạn

nốt Rê

Siêu tham số	Phân loại nhị phân	Phân loại nhị phân đa nhãn	Phân loại đa lớp
--------------	--------------------	----------------------------	------------------

# các lớp ẩn	Tùy thuộc vào nhiệm vụ, nhưng thông thường là từ 1 đến 5.		
# nơ-ron đầu ra	1	1 nhãn nhị phân	1 cái/lớp
Kích hoạt lớp đầu ra	Sigmoid	Hình chữ S	Softmax
Hàm mất mát	entropy x	entropy x	entropy x

MẸO

Trước khi tiếp tục, tôi khuyên bạn nên làm bài tập 1 ở cuối chương này. Bạn sẽ thử nghiệm với nhiều kiến trúc mạng nơ-ron khác nhau và trực quan hóa kết quả đầu ra của chúng bằng cách sử dụng... TensorFlow Playground. Công cụ này sẽ rất hữu ích để hiểu rõ hơn về MLP, bao gồm... ảnh hưởng của tất cả các siêu tham số (số lớp và số nơ-ron, kích hoạt) chức năng, và hơn thế nữa).

Giờ bạn đã nắm được tất cả các khái niệm cần thiết để bắt đầu triển khai MLP. Keras!

Triển khai MLP bằng Keras

Keras là API học sâu cấp cao của TensorFlow: nó cho phép bạn xây dựng, Huấn luyện, đánh giá và thực thi tất cả các loại mạng nơ-ron. Keras gốc Thư viện này được phát triển bởi François Chollet như một phần của dự án nghiên cứu. và được phát hành như một dự án mã nguồn mở độc lập vào tháng 3 năm 2015. Nó nhanh chóng trở nên phổ biến nhờ tính dễ sử dụng, tính linh hoạt và vẻ ngoài đẹp mắt. thiết kế.

GHI CHÚ

Trước đây, Keras hỗ trợ nhiều backend, bao gồm TensorFlow, PlaidML, Theano và Microsoft Cognitive Toolkit (CNTK) (hai backend cuối cùng đáng tiếc đã bị loại bỏ), nhưng kể từ phiên bản 2.4, Keras chỉ hỗ trợ TensorFlow. Ngược lại, TensorFlow trước đây bao gồm nhiều API cấp cao, nhưng Keras đã chính thức được chọn làm API cấp cao ưu tiên khi TensorFlow 2 ra mắt. Việc cài đặt TensorFlow cũng sẽ tự động cài đặt Keras, và Keras sẽ không hoạt động nếu không có TensorFlow được cài đặt. Tóm lại, Keras và TensorFlow đã "yêu nhau" và "kết hôn". François Chollet đã gia nhập Google và tiếp tục dẫn dắt dự án Keras. Các thư viện Deep Learning phổ biến khác bao gồm [PyTorch của Facebook](#) và [JAX của Google](#). ¹³

Được rồi, giờ chúng ta hãy sử dụng Keras! Chúng ta sẽ bắt đầu bằng cách xây dựng một mạng nơ-ron đa lớp (MLP) để phân loại hình ảnh.

GHI CHÚ

Colab Runtimes được cài đặt sẵn các phiên bản TensorFlow và Keras mới nhất. Tuy nhiên, nếu bạn muốn cài đặt chúng trên máy tính của mình, vui lòng xem hướng dẫn cài đặt tại <https://hom1.info/install>.

Xây dựng bộ phân loại hình ảnh sử dụng API tuần tự

Đầu tiên, chúng ta cần tải một tập dữ liệu. Chúng ta sẽ sử dụng Fashion MNIST, một tập dữ liệu thay thế trực tiếp cho MNIST (đã được giới thiệu trong [Chương 3](#)). Nó có định dạng giống hệt MNIST (70.000 hình ảnh thang độ xám, mỗi hình có kích thước 28 × 28 pixel, với 10 lớp), nhưng các hình ảnh này đại diện cho các mặt hàng thời trang chứ không phải chữ số viết tay, do đó mỗi lớp đa dạng hơn, và bài toán trở nên khó khăn hơn đáng kể so với MNIST. Ví dụ, một mô hình tuyến tính đơn giản đạt độ chính xác khoảng 92% trên MNIST, nhưng chỉ khoảng 83% trên Fashion MNIST.

Sử dụng Keras để tải tập dữ liệu.

Keras cung cấp một số hàm tiện ích để tìm nạp và tải các tập dữ liệu phổ biến, bao gồm MNIST, Fashion MNIST và một vài tập dữ liệu khác. Hãy tải Fashion MNIST. Tập dữ liệu này đã được xáo trộn và chia thành tập huấn luyện (60.000 hình ảnh).

và một tập dữ liệu thử nghiệm (10.000 hình ảnh), nhưng chúng ta hãy giữ lại 5.000 hình ảnh cuối cùng từ tập dữ liệu huấn luyện để xác thực:

```
import tensorflow as tf

fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
```

MẸO

TensorFlow thường được nhập dưới dạng `tf`, và API của Keras có sẵn thông qua `tf.keras`.

Khi tải tập dữ liệu MNIST hoặc Fashion MNIST bằng Keras thay vì Scikit-Learn, một điểm khác biệt quan trọng là mỗi hình ảnh được biểu diễn dưới dạng mảng 28×28 thay vì mảng 1 chiều có kích thước 784. Hơn nữa, cường độ pixel được biểu diễn dưới dạng số nguyên (từ 0 đến 255) thay vì số thực (từ 0,0 đến 255,0). Hãy cùng xem xét hình dạng và kiểu dữ liệu của tập huấn luyện:

```
>>> X_train.shape
(55000, 28, 28)
>>> X_train.dtype
dtype('uint8')
```

Để đơn giản, chúng ta sẽ giảm cường độ pixel xuống phạm vi 0-1 bằng cách chia chúng cho 255,0 (thao tác này cũng chuyển đổi chúng thành số thực):

```
X_train, X_valid, X_test = X_train / 255, X_valid / 255, X_test / 255.
```

Với MNIST, khi nhãn bằng 5, điều đó có nghĩa là hình ảnh thể hiện chữ số 5 viết tay.

Khá đơn giản. Tuy nhiên, đối với Fashion MNIST, chúng ta cần danh sách tên lớp để biết mình đang làm việc với cái gì:

```
class_names = ["Áo phông/Áo thun", "Quần", "Áo chui đầu", "Váy",
               "Áo choàng",
```

[Dép xăng đan, áo sơ mi, giày thể thao, túi xách, boots cổ ngắn]

Ví dụ, hình ảnh đầu tiên trong tập dữ liệu huấn luyện thể hiện một đôi boots cổ thấp:

```
>>> class_names[y_train[0]]  
'Giày boots cổ thấp'
```

Hình 10-10 hiển thị một số mẫu từ tập dữ liệu Fashion MNIST.

Ankle boot T-shirt/top T-shirt/top Dress T-shirt/top Pullover Sneaker Pullover Sandal Sandal



T-shirt/top Ankle boot Sandal Sandal Sneaker Ankle boot Trouser T-shirt/top Shirt Coat



Dress Trouser Coat Bag Coat Dress T-shirt/top Pullover Coat Coat



Sandal Dress Shirt Shirt T-shirt/top Bag Sandal Pullover Trouser Shirt



Hình 10-10. Các mẫu từ bộ sưu tập thời trang MNIST.

Tạo mô hình bằng cách sử dụng API Sequential

Bây giờ chúng ta hãy xây dựng mạng nơ-ron! Đây là một mạng nơ-ron đa lớp (MLP) phân loại với hai lớp ẩn:

```
tf.random.set_seed(42)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(300, activation="relu"))
model.add(tf.keras.layers.Dense(100, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

Chúng ta hãy cùng xem xét từng dòng mã này:

- Đầu tiên, chúng ta thiết lập hạt giống ngẫu nhiên của TensorFlow để đảm bảo kết quả có thể tái tạo: trọng số ngẫu nhiên của các lớp ẩn và lớp đầu ra sẽ giống nhau mỗi khi bạn chạy notebook. Bạn cũng có thể chọn sử dụng hàm `tf.keras.utils.set_random_seed()`, hàm này thiết lập hạt giống ngẫu nhiên một cách thuận tiện cho TensorFlow, Python (`random.seed()`) và NumPy (`np.random.seed()`).
- Dòng lệnh tiếp theo tạo ra một mô hình Sequential. Đây là loại mô hình Keras đơn giản nhất dành cho mạng nơ-ron, chỉ bao gồm một chồng các lớp được kết nối tuần tự. Điều này được gọi là API Sequential.
- Tiếp theo, chúng ta xây dựng lớp đầu tiên và thêm nó vào mô hình: đó là lớp Đầu vào. Chúng ta chỉ định hình dạng đầu vào, không bao gồm kích thước lô, chỉ bao gồm hình dạng của các trường hợp. Keras cần biết hình dạng của đầu vào để có thể xác định hình dạng của ma trận trọng số kết nối của lớp ẩn đầu tiên.
- Tiếp theo, chúng ta thêm một lớp Flatten có vai trò chuyển đổi mỗi ảnh đầu vào thành một mảng 1 chiều: ví dụ, nếu nó nhận được một lô ảnh có kích thước `[32, 28, 28]`, nó sẽ định hình lại thành `[32, 784]`. Nói cách khác, nếu nó nhận được dữ liệu đầu vào `X`, nó sẽ tính toán `X.reshape(-1, 784)`. Lớp này thực hiện

Nó không có bất kỳ tham số nào; nó chỉ được dùng để thực hiện một số bước tiền xử lý đơn giản.

- Tiếp theo, chúng ta thêm một lớp ẩn Dense với 300 nơ-ron. Lớp này sẽ sử dụng hàm kích hoạt ReLU. Mỗi lớp Dense quản lý ma trận trọng số riêng, chứa tất cả các trọng số kết nối giữa các nơ-ron và đầu vào của chúng. Nó cũng quản lý một vectơ các hệ số thiên vị (một hệ số cho mỗi nơ-ron).
Khi nhận được dữ liệu đầu vào, nó sẽ tính toán theo **Phương trình 10-2**.
- Tiếp theo, chúng ta thêm một lớp ẩn Dense thứ hai với 100 nơ-ron, cũng sử dụng hàm kích hoạt ReLU.
- Cuối cùng, chúng ta thêm một lớp đầu ra Dense với 10 nơ-ron (một nơ-ron cho mỗi lớp), sử dụng hàm kích hoạt softmax vì các lớp này là độc lập với nhau.

MẸO

Việc chỉ định `activation="relu"` tương đương với việc chỉ định `activation=tf.keras.activations.relu`. Các hàm kích hoạt khác là...

Các hàm này có sẵn trong gói `tf.keras.activations`, và chúng ta sẽ sử dụng nhiều hàm trong số đó trong cuốn sách này.

Xem thêm tại <https://keras.io/api/layers/activations/> Để xem danh sách đầy đủ, hãy tham khảo thêm. Chúng ta cũng sẽ định nghĩa các hàm kích hoạt tùy chỉnh của riêng mình trong **Chương 12**.

Thay vì thêm từng lớp một như chúng ta vừa làm, thường thì việc truyền một danh sách các lớp khi tạo mô hình Sequential sẽ thuận tiện hơn.

Bạn cũng có thể bỏ lớp Input và thay vào đó chỉ định `input_shape` ở lớp đầu tiên:

```
model =  
    tf.keras.Sequential([ tf.keras.layers.Flatten(input_shape=[28,  
28]), tf.keras.layers.Dense(300, activation="relu"),  
    tf.keras.layers.Dense(100, activation="relu"),  
    tf.keras.layers.Dense(10, activation="softmax")  
])
```

Phương thức `summary()` của mô hình hiển thị tất cả các lớp của mô hình.

bao gồm tên của từng lớp (tên này được tạo tự động trừ khi bạn...).

đặt nó khi tạo lớp), hình dạng đầu ra của nó (None có nghĩa là kích thước lô

có thể là bất cứ thứ gì), và số lượng tham số của nó. Phần tóm tắt kết thúc bằng

Tổng số tham số, bao gồm cả tham số có thể huấn luyện và không thể huấn luyện.

các tham số. Ở đây chúng ta chỉ có các tham số có thể huấn luyện được (chúng ta sẽ thấy một số tham số không thể huấn luyện được ở phần sau của chương này):

```
>>> model.summary()
```

```
Mô hình: "tuần tự"
```

Lớp (loại)	Hình dạng đầu ra	Tham số #
=====	=====	=====
làm phẳng (Flatten)	(Không có, 784)	0
dày đặc (Dense)	(Không có, 300)	235500
dense_1 (Dense)	(Không có, 100)	30100
dense_2 (Dense)	(Không có, 10)	1010
=====	=====	=====
Tổng số tham số: 266.610		
Số tham số có thể huấn luyện: 266.610		
Các tham số không thể huấn luyện: 0		

Lưu ý rằng các lớp Dense thường có rất nhiều tham số. Ví dụ,

Lớp ẩn đầu tiên có 784×300 trọng số kết nối, cộng thêm 300 số hạng thiên vị.

Điều này tương đương với 235.500 tham số! Điều đó mang lại cho mô hình khá nhiều...

Tính linh hoạt để phù hợp với dữ liệu huấn luyện, nhưng điều đó cũng có nghĩa là mô hình chạy...

Nguy cơ quá khớp dữ liệu, đặc biệt khi bạn không có nhiều dữ liệu huấn luyện.

Chúng ta sẽ quay lại vấn đề này sau.

Tất cả các lớp trong một mô hình phải có tên duy nhất (ví dụ: "dense_2"). Bạn có thể

Đặt tên lớp một cách rõ ràng bằng cách sử dụng đối số `name`` của hàm tạo, nhưng

Thông thường, sẽ đơn giản hơn nếu để Keras tự động đặt tên cho các lớp, vì chúng ta chỉ cần...

đã làm: Keras lấy tên lớp của lớp và chuyển đổi nó sang dạng snake case (ví dụ: a)

Lớp từ lớp `MyCoolLayer` được đặt tên là "my_cool_layer" bởi

mặc định). Hơn nữa, Keras đảm bảo rằng tên đó là duy nhất trên toàn cầu, ngay cả khi

Trên các mô hình khác nhau, bằng cách thêm chỉ mục nếu cần, như trong "dense_2". Nhưng tại sao, bạn có thể hỏi, lại phải làm cho tên các mô hình khác nhau là duy nhất? Vâng, điều này giúp việc hợp nhất các mô hình trở nên dễ dàng hơn mà không gây ra xung đột tên.

MẸO

Tất cả trạng thái toàn cục do Keras quản lý đều được lưu trữ trong một phiên Keras, mà bạn có thể xóa bằng cách sử dụng `tf.keras.backend.clear_session()`. Cụ thể, thao tác này đặt lại tên. quầy.

Bạn có thể dễ dàng lấy danh sách các lớp của một mô hình bằng cách sử dụng thuộc tính `layers`, hoặc sử dụng phương thức `get_layer()` để truy cập một lớp theo tên:

```
>>> model.layers
[<keras.layers.core.flatten.Flatten at 0x7fa1dea02250>, <keras.layers.core.dense.Dense
at 0x7fa1c8f42520>, <keras.layers.core.dense.Dense at 0x7fa188be7ac0>,
<keras.layers.core.dense.Dense at 0x7fa188be7fa0>] >>> hidden1 =
model.layers[1] >>> hidden1.name 'dense' >>> model.get_layer('dense') is
hidden1 True
```

Tất cả các tham số của một lớp có thể được truy cập bằng cách sử dụng các phương thức `get_weights()` và `set_weights()` của nó. Đối với lớp `Dense`, điều này bao gồm cả trọng số kết nối và các số hạng bias:

```
>>> weights, biases = hidden1.get_weights() >>>
weights
array([[ 0.02448617, -0.00877795, -0.02189048, ..., 0.03859074,
-0.06889391],
[ 0.00476504, -0.03105379, -0.0586676, ..., -0.02763776,
-0.04165364],
...,
[ 0.07061854, -0.06960931, 0.07038955, ..., 0.00034875,
0.02878492],
[-0.06022581, 0.01577859, -0.02585464, ..., 0.00272203,
```

```

-0.06793761]],
        dtype=float32) >>>
weights.shape (784,
300) >>>
biases
array([0., 0., 0., 0., 0., 0., 0., 0., 0., ..., 0., 0., 0.], dtype=float32) >>>
biases.shape (300,)

```

Lưu ý rằng lớp Dense đã khởi tạo các trọng số kết nối một cách ngẫu nhiên (điều này cần thiết để phá vỡ tính đối xứng, như chúng ta đã thảo luận trước đó), và các hệ số bias được khởi tạo bằng 0, điều này là ổn. Nếu bạn muốn sử dụng phương pháp khởi tạo khác, bạn có thể đặt `kernel_initializer` (kernel là tên gọi khác của ma trận trọng số kết nối) hoặc `bias_initializer` khi tạo lớp. Chúng ta sẽ thảo luận thêm về các bộ khởi tạo trong [Chương 11](#), nhưng nếu bạn muốn xem danh sách đầy đủ, hãy truy cập <https://keras.io/api/layers/initializers/>.

GHI CHÚ

Hình dạng của ma trận trọng số phụ thuộc vào số lượng đầu vào, đó là lý do tại sao chúng ta chỉ định `input_shape` khi tạo mô hình. Nếu bạn không chỉ định hình dạng đầu vào, điều đó cũng không sao: Keras sẽ chỉ đợi cho đến khi nó biết hình dạng đầu vào trước khi thực sự xây dựng các tham số của mô hình. Điều này sẽ xảy ra khi bạn cung cấp cho nó một số dữ liệu (ví dụ: trong quá trình huấn luyện), hoặc khi bạn gọi phương thức `build()` của nó. Cho đến khi các tham số của mô hình được xây dựng, bạn sẽ không thể thực hiện một số thao tác nhất định, chẳng hạn như hiển thị tóm tắt mô hình hoặc lưu mô hình. Vì vậy, nếu bạn biết hình dạng đầu vào khi tạo mô hình, tốt nhất là nên chỉ định nó.

Biên dịch mô hình Sau khi

mô hình được tạo, bạn phải gọi phương thức `compile()` của nó để chỉ định hàm mất mát và trình tối ưu hóa cần sử dụng. Tùy chọn, bạn có thể chỉ định một danh sách các chỉ số bổ sung cần tính toán trong quá trình huấn luyện và đánh giá:

```

model.compile(loss="sparse_categorical_crossentropy", optimizer="sgd",
              metrics=["accuracy"])

```

GHI CHÚ

Việc sử dụng `loss="sparse_categorical_crossentropy"` tương đương với việc sử dụng `loss=tf.keras.losses.sparse_categorical_crossentropy`.

Tương tự, việc chỉ định `optimizer="sgd"` tương đương với việc chỉ định `optimizer=tf.keras.optimizers.SGD()`, và `metrics=["accuracy"]` tương đương với `metrics=`

`[tf.keras.metrics.sparse_categorical_accuracy]` (khi sử dụng hàm mất mát này). Chúng ta sẽ sử dụng nhiều hàm mất mát, trình tối ưu hóa và số liệu khác trong cuốn sách này; để xem danh sách đầy đủ, hãy truy cập <https://keras.io/api/losses/>, <https://keras.io/api/optimizers/>, và <https://keras.io/api/metrics/>.

Đoạn mã này cần một vài giải thích. Đầu tiên, chúng ta sử dụng

hàm mất mát `"sparse_categorical_crossentropy"` vì chúng ta có nhãn thưa (nghĩa là, đối với mỗi trường hợp, chỉ có một chỉ số lớp mục tiêu, từ 0 đến 9 trong trường hợp này), và các lớp là độc lập. Nếu thay vào đó chúng ta có một xác suất mục tiêu cho mỗi lớp đối với mỗi trường hợp (chẳng hạn như vectơ one-hot, ví dụ: `[0., 0., 0., 1., 0., 0., 0., 0., 0., 0.]` để biểu diễn lớp 3), thì chúng ta cần sử dụng hàm mất mát `"categorical_crossentropy"` thay thế. Nếu chúng ta đang thực hiện phân loại nhị phân hoặc phân loại nhị phân đa nhãn, thì chúng ta sẽ sử dụng hàm kích hoạt `"sigmoid"` trong lớp đầu ra thay vì hàm kích hoạt `"softmax"`, và chúng ta sẽ sử dụng hàm mất mát `"binary_crossentropy"`.

MẸO

Nếu bạn muốn chuyển đổi nhãn thưa (tức là chỉ số lớp) thành nhãn vectơ one-hot, hãy sử dụng hàm `tf.keras.utils.to_categorical()`. Để làm ngược lại, hãy sử dụng hàm `np.argmax()` với `axis=1`.

Về bộ tối ưu hóa, `"sgd"` có nghĩa là chúng ta sẽ huấn luyện mô hình bằng phương pháp Giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent). Nói cách khác, Keras sẽ thực hiện thuật toán lan truyền ngược đã mô tả trước đó (tức là phương pháp tự động vi phân ngược).

(cộng thêm thuật toán Gradient Descent). Chúng ta sẽ thảo luận về các thuật toán tối ưu hiệu quả hơn trong [Chương 11](#). Chúng cải thiện Gradient Descent, chứ không phải autodiff.

GHI CHÚ

Khi sử dụng trình tối ưu hóa SGD, việc điều chỉnh tốc độ học là rất quan trọng. Vì vậy, bạn thường muốn sử dụng `optimizer=tf.keras.optimizers.SGD(learning_rate=???)` để thiết lập tốc độ học, thay vì `optimizer="sgd"`, mặc định là tốc độ học 0.01.

Cuối cùng, vì đây là một bộ phân loại, nên việc đo lường độ chính xác của nó trong quá trình huấn luyện và đánh giá là rất hữu ích, đó là lý do tại sao chúng ta đặt `metrics=["accuracy"]`.

Đào tạo và đánh giá mô hình

Giờ mô hình đã sẵn sàng để huấn luyện. Để làm điều này, chúng ta chỉ cần gọi phương thức `fit()` của nó:

```
>>> history = model.fit(X_train, y_train, epochs=30,
...                       validation_data=(X_valid, y_valid))
...
Epoch 1/30
1719/1719 [=====] - 2s 989us/step -
  loss: 0.7220 - sparse_categorical_accuracy: 0.7649 -
  val_loss: 0.4959 - val_sparse_categorical_accuracy: 0.8332 Epoch
2/30
1719/1719 [=====] - 2s 964us/step -
  loss: 0.4825 - sparse_categorical_accuracy: 0.8332 -
  val_loss: 0.4567 - val_sparse_categorical_accuracy: 0.8384 [...]

Epoch 30/30
1719/1719 [=====] - 2s 963us/step -
  loss: 0.2235 - sparse_categorical_accuracy: 0.9200 -
  val_loss: 0.3056 - val_sparse_categorical_accuracy: 0.8894
```

Chúng ta truyền vào đó các đặc trưng đầu vào (`X_train`) và các lớp mục tiêu (`y_train`), cũng như số lượng epoch để huấn luyện (nếu không, nó sẽ mặc định là 1, điều này chắc chắn sẽ không đủ để hội tụ đến một giải pháp tốt).

Bạn cũng có thể truyền thêm tập dữ liệu kiểm chứng (tùy chọn). Keras sẽ đo lường tổn thất và các chỉ số bổ sung trên tập dữ liệu này vào cuối mỗi epoch, điều này rất hữu ích để xem mô hình thực sự hoạt động tốt như thế nào. Nếu hiệu suất trên tập huấn luyện tốt hơn nhiều so với trên tập kiểm chứng, mô hình của bạn có thể đang bị quá khớp với tập huấn luyện, hoặc có lỗi, chẳng hạn như sự không khớp dữ liệu giữa tập huấn luyện và tập kiểm chứng.

MẸO

Lỗi về hình dạng khá phổ biến, đặc biệt là khi mới bắt đầu, vì vậy bạn nên làm quen với các thông báo lỗi: hãy thử huấn luyện một mô hình với đầu vào và/hoặc nhãn có hình dạng sai và xem các lỗi bạn nhận được. Tương tự, hãy thử biên dịch mô hình với `loss="categorical_crossentropy"` thay vì `loss="sparse_categorical_crossentropy"`, hoặc loại bỏ lớp Flatten.

Và thế là xong! Mạng nơ-ron đã được huấn luyện. Ở mỗi epoch trong quá trình huấn luyện, Keras hiển thị số lượng mini-batch đã được xử lý cho đến nay, ở phía bên trái của thanh tiến trình. Kích thước batch mặc định là 32, và vì tập huấn luyện có 55.000 hình ảnh, mô hình sẽ xử lý 1.719 batch mỗi epoch: 1.718 batch có kích thước 32 và một batch có kích thước 24. Sau thanh tiến trình, bạn có thể thấy thời gian huấn luyện trung bình cho mỗi mẫu, và tổn thất cũng như độ chính xác (hoặc bất kỳ chỉ số bổ sung nào khác mà bạn yêu cầu) trên cả tập huấn luyện và tập kiểm chứng. Hãy lưu ý rằng tổn thất huấn luyện đã giảm xuống, đây là một dấu hiệu tốt, và độ chính xác xác thực đạt 88,94% sau 30 epoch. Con số này thấp hơn một chút so với độ chính xác huấn luyện, vì vậy có một chút hiện tượng quá khớp xảy ra, nhưng không đáng kể.

MẸO

Thay vì truyền tập dữ liệu kiểm chứng bằng đối số ``validation_data``, bạn có thể đặt ``validation_split`` thành tỷ lệ của tập dữ liệu huấn luyện mà bạn muốn Keras sử dụng để kiểm chứng. Ví dụ, ``validation_split=0.1`` cho Keras biết sử dụng 10% dữ liệu cuối cùng (trước khi xáo trộn) để kiểm chứng.

Nếu tập dữ liệu huấn luyện bị lệch nhiều, với một số lớp được đại diện quá mức và một số lớp khác bị đại diện dưới mức, thì việc thiết lập đối số `'class_weight'` khi gọi phương thức `'fit()'` sẽ rất hữu ích, để gán trọng số lớn hơn cho các lớp bị đại diện dưới mức và trọng số nhỏ hơn cho các lớp được đại diện quá mức. Keras sẽ sử dụng các trọng số này khi tính toán tổn thất. Nếu bạn cần trọng số cho từng trường hợp riêng lẻ, hãy thiết lập đối số `'sample_weight'`. Nếu cả `'class_weight'` và `'sample_weight'` đều được cung cấp, thì Keras sẽ nhân chúng lại với nhau. Trọng số cho từng trường hợp riêng lẻ có thể hữu ích, ví dụ, nếu một số trường hợp được gán nhãn bởi các chuyên gia trong khi những trường hợp khác được gán nhãn bằng nền tảng crowdsourcing: bạn có thể muốn gán trọng số lớn hơn cho trường hợp đầu tiên. Bạn cũng có thể cung cấp trọng số mẫu (nhưng không phải trọng số lớp) cho tập dữ liệu xác thực bằng cách thêm chúng làm mục thứ ba trong bộ dữ liệu `'validation_data'`.

Phương thức `fit()` trả về một đối tượng History chứa các tham số huấn luyện (`history.params`), danh sách các epoch đã trải qua (`history.epoch`), và quan trọng nhất là một từ điển (`history.history`) chứa giá trị loss và các chỉ số bổ sung được đo lường ở cuối mỗi epoch trên tập huấn luyện và trên tập kiểm định (nếu có).

Nếu bạn sử dụng từ điển này để tạo một Pandas DataFrame và gọi phương thức `plot()` của nó, bạn sẽ nhận được các đường cong học tập được hiển thị trong **Hình 10-11**:

```
import matplotlib.pyplot as plt
import pandas as pd

pd.DataFrame(history.history).plot( figsize=(8, 5), xlim=[0,
29], ylim=[0,
1], grid=True, xlabel="Epoch",
style=["r--", "r--.", "b-", "b-*"]) plt.show()
```

